Computer Vision —— Object Recognition

2026/9/29

桑 农

Slides from CV course with R. Fergus, and S. Savarese, S. Lazebnik, D. Lowe, P. Viola

Agenda

- Introduction
- Objects & Faces recognition
- Bag-of-words models
- Part-based models

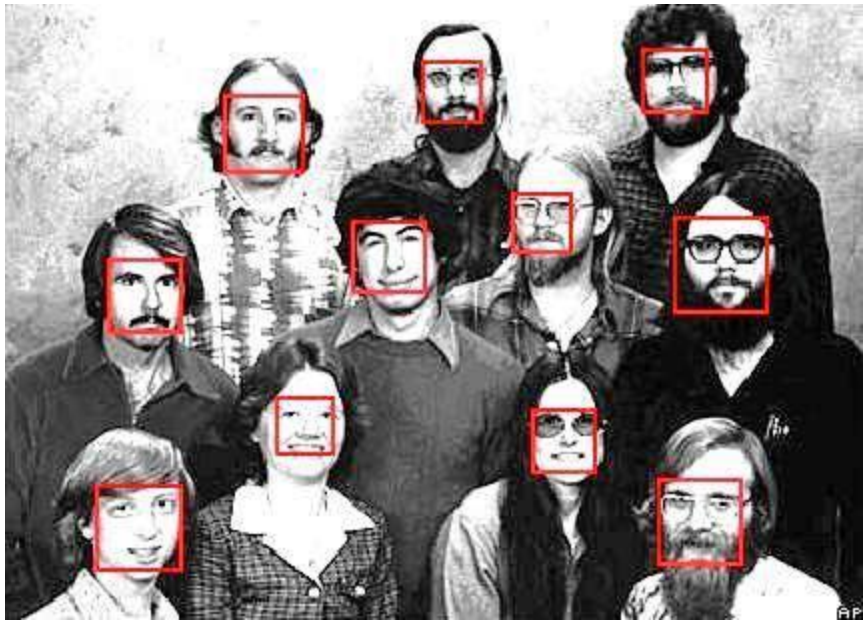
Overview

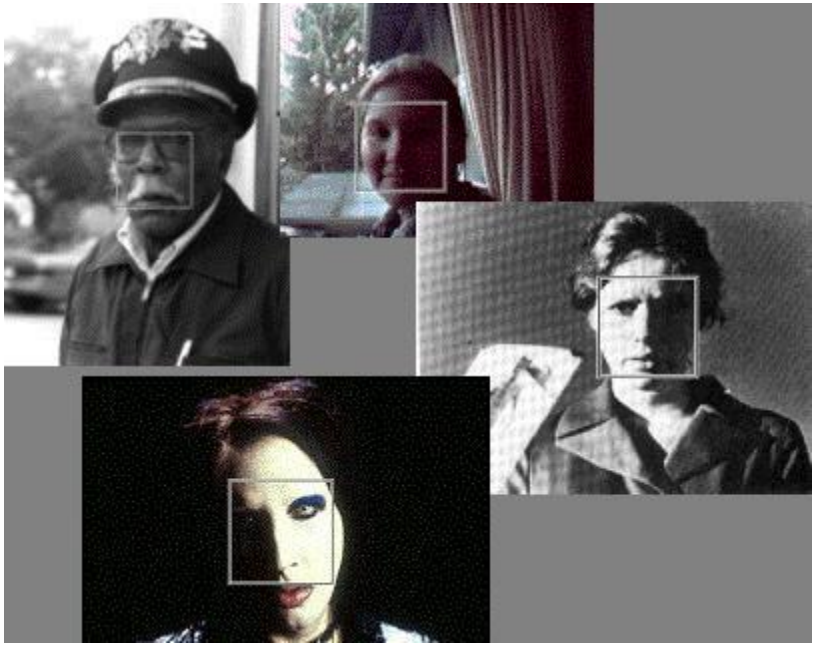
- Specific Object Recognition
  - Lowe‘04
  - Rothganger et al.‘03
- Face Dectection
  - Sub-space methods
  - Neural Network-based approaches
  - Viola & Jones
- Face Recognition

Face Detection & Recognition

Slides from
P. Viola, S. Lazebnik, A. Torralba

Face detection







Face detection and recognition



Detection



Recognition

“Sally”

Consumer application: iPhoto 2009

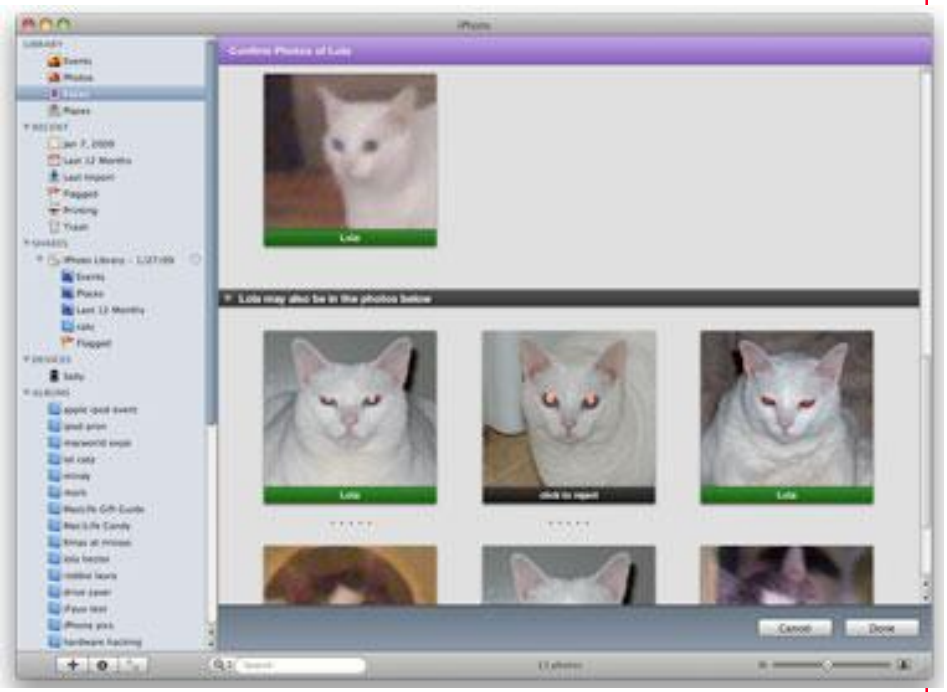


http://www.apple.com/ilife/iphoto/

Consumer application: iPhoto 2009

- Can be trained to recognize pets!





http://www.maclife.com/article/news/iphotos_faces_recognizes_cats

Challenges:

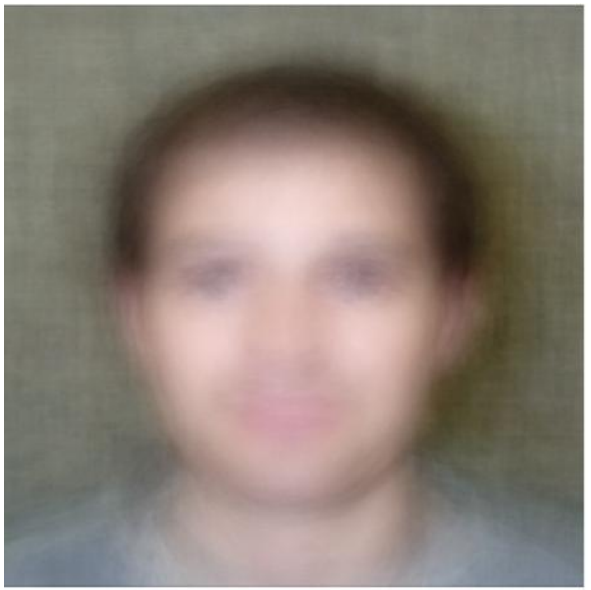
- Variability due to:

  Intra-class variation (but fairly small)

  Illumination

  Occlusions (eyeglasses, facial hair, hair)

  Fairly limited viewpoint (frontal, profile)



Practical Challenges of face detection

- Sliding window detector must evaluate tens of thousands of location/scale combinations
- Faces are rare: 0–10 per image

  For computational efficiency, we should try to spend as little time as possible on the non-face windows

  A megapixel image has ~10^6 pixels and a comparable number of candidate face locations

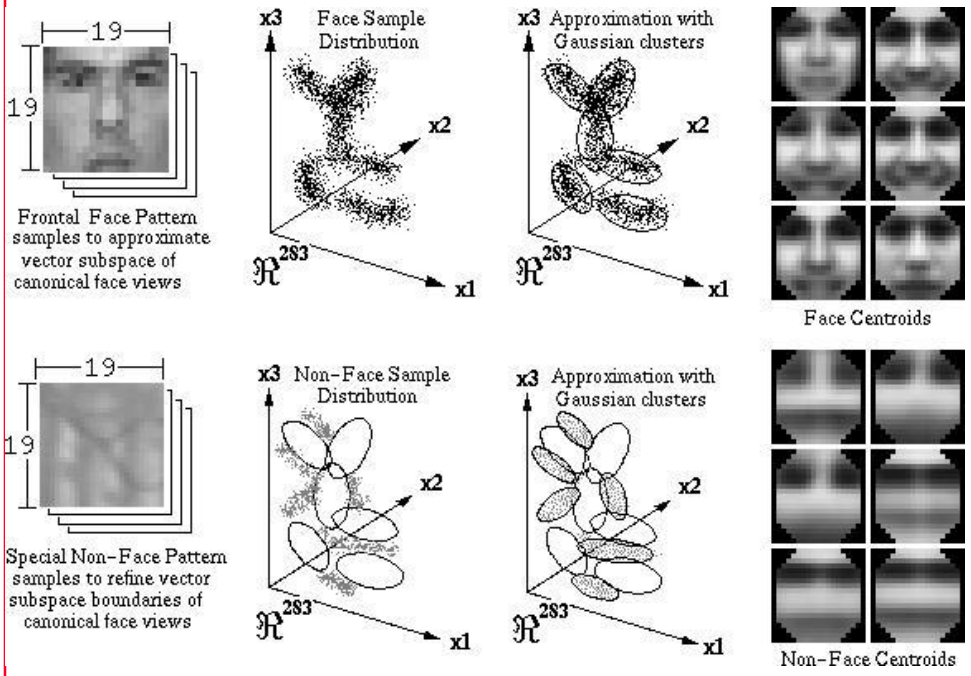
  To avoid having a false positive in every image, our false positive rate has to be less than 10^{-6}

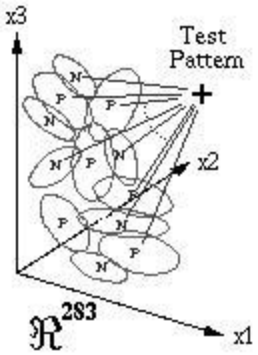
Overview

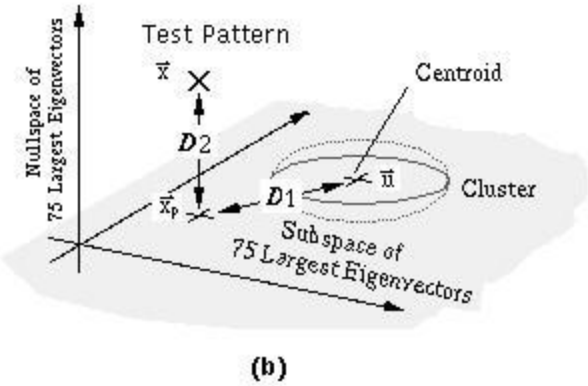
- Specific Object Recognition
  - Lowe‘04
  - Rothganger et al.‘03
- Face Dectection
  - Sub-space methods
  - Neural Network-based approaches
  - Viola & Jones
- Face Recognition

Distribution-Based Face Detector

- Learn face and nonface models from examples [Sung and Poggio 95]
  - Cluster and project the examples to a lower dimensional space using Gaussian distributions and PCA
  - Detect faces using distance metric to face and nonface clusters







Distribution-Based Face Detector

- Learn face and nonface models from examples [Sung and Poggio 95]

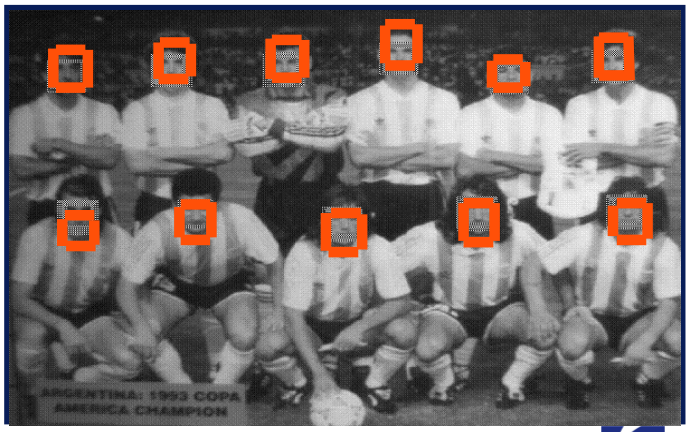




Training Database

1000+ Real, 3000+ *VIRTUAL*

50,0000+ Non-Face Pattern

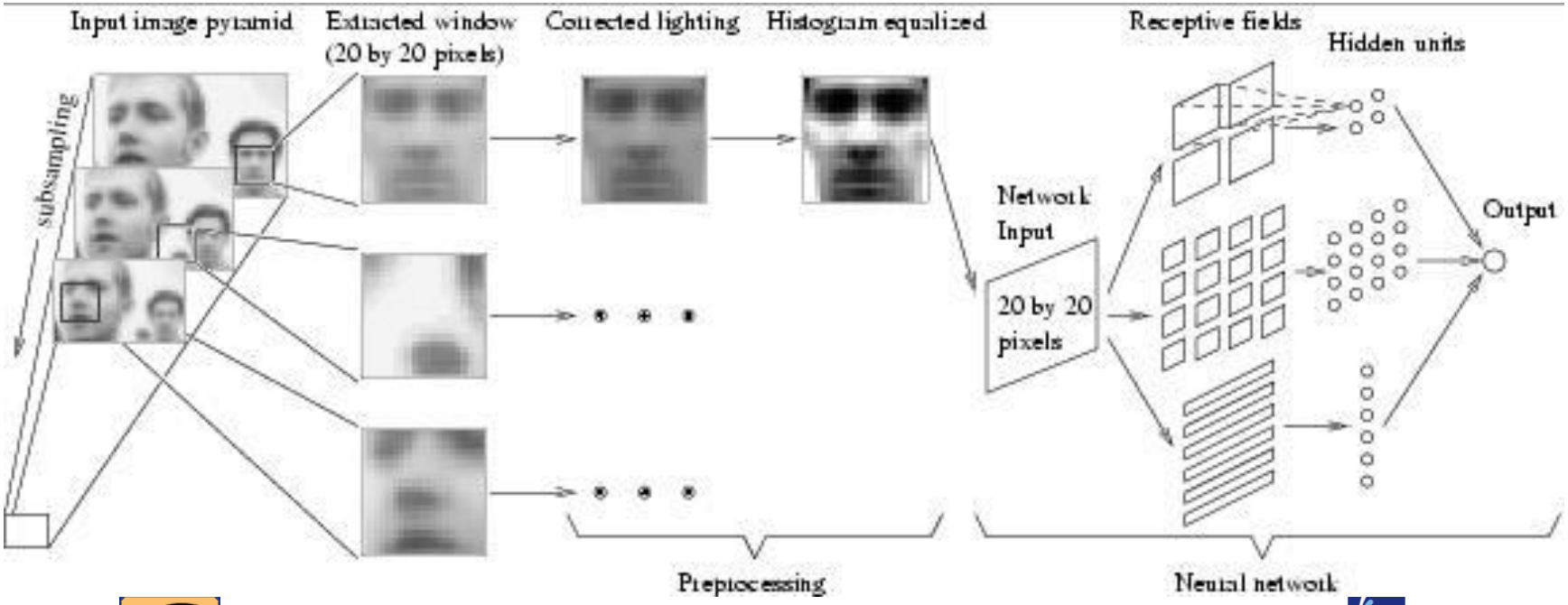


Overview

- Specific Object Recognition
  - Lowe‘04
  - Rothganger et al.‘03
- Face Dectection
  - Sub-space methods
  - Neural Network-based approaches
  - Viola & Jones
- Face Recognition

Neural Network-Based Face Detector

- Train a set of multilayer perceptrons and arbitrate a decision among all outputs [Rowley et al. 98]



Oval mask for ignoring background pixels:



Original window:



Best fit linear function:



Lighting corrected window: (linear function subtracted)



Histogram equalized window:



The steps in preprocessing a window. First, a linear function is fit to the intensity values in the window, and then subtracted out, correcting for some extreme lighting conditions. Then, histogram equalization is applied, to correct for different camera gains and to improve contrast. For each of these steps, the mapping is computed based on pixels inside the oval mask, while the mapping is applied to the entire window.

From: http://www.ius.cs.cmu.edu/IUS/har2/har/www/CMU-CS-95-158R/

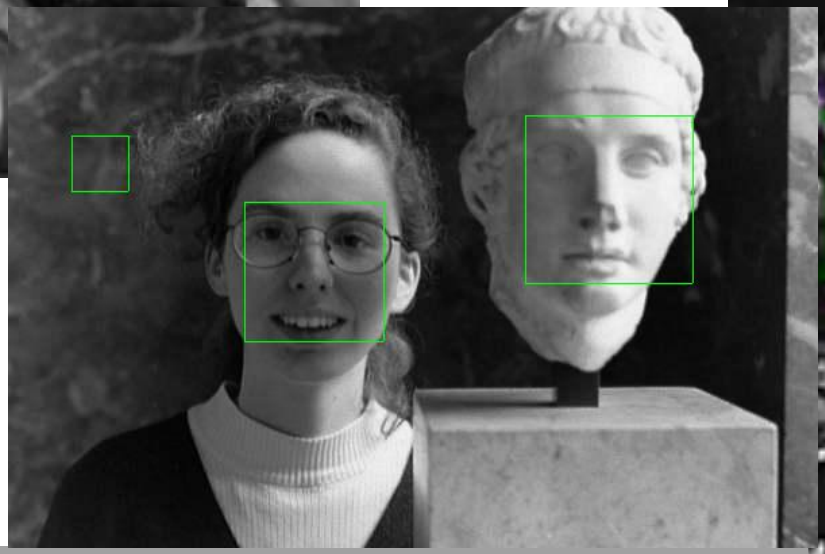
Example CMU face detector results

input



All images from: http://www.ius.cs.cmu.edu/demos/facedemo.html






STAR TREK
DEEP SPACE NINE
THE WAY OF THE WARRIOR

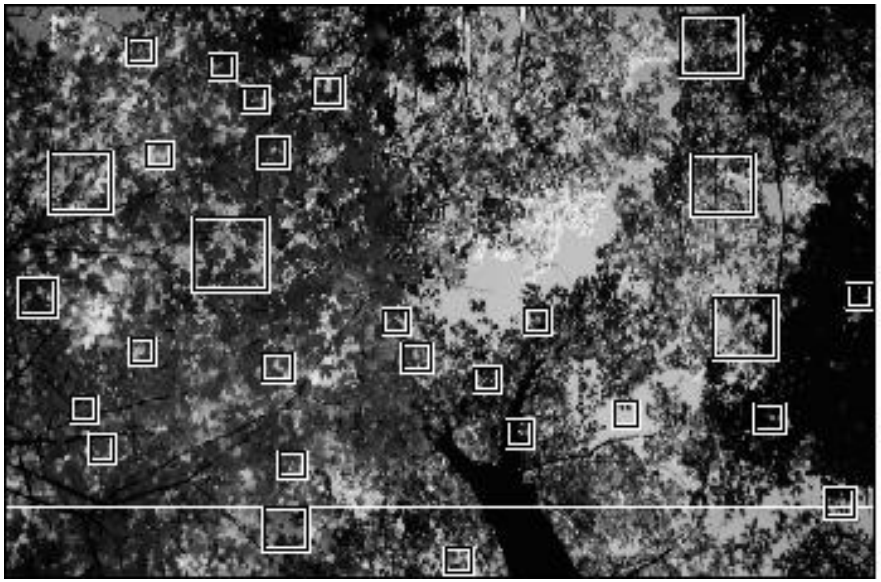


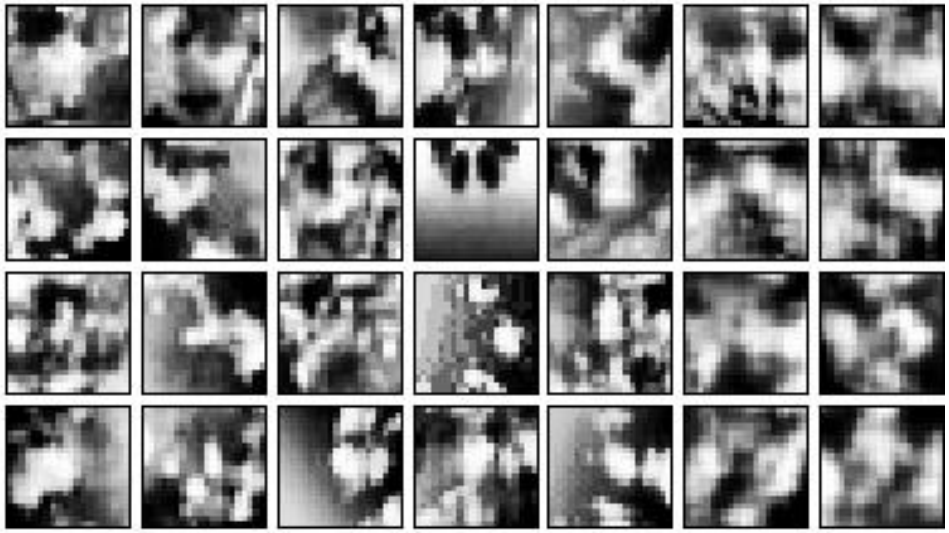
Example face images, randomly mirrored, rotated, translated, and scaled by small amounts (photos are of the three authors).



From: http://www.ius.cs.cmu.edu/IUS/har2/har/www/CMU-CS-95-158R/

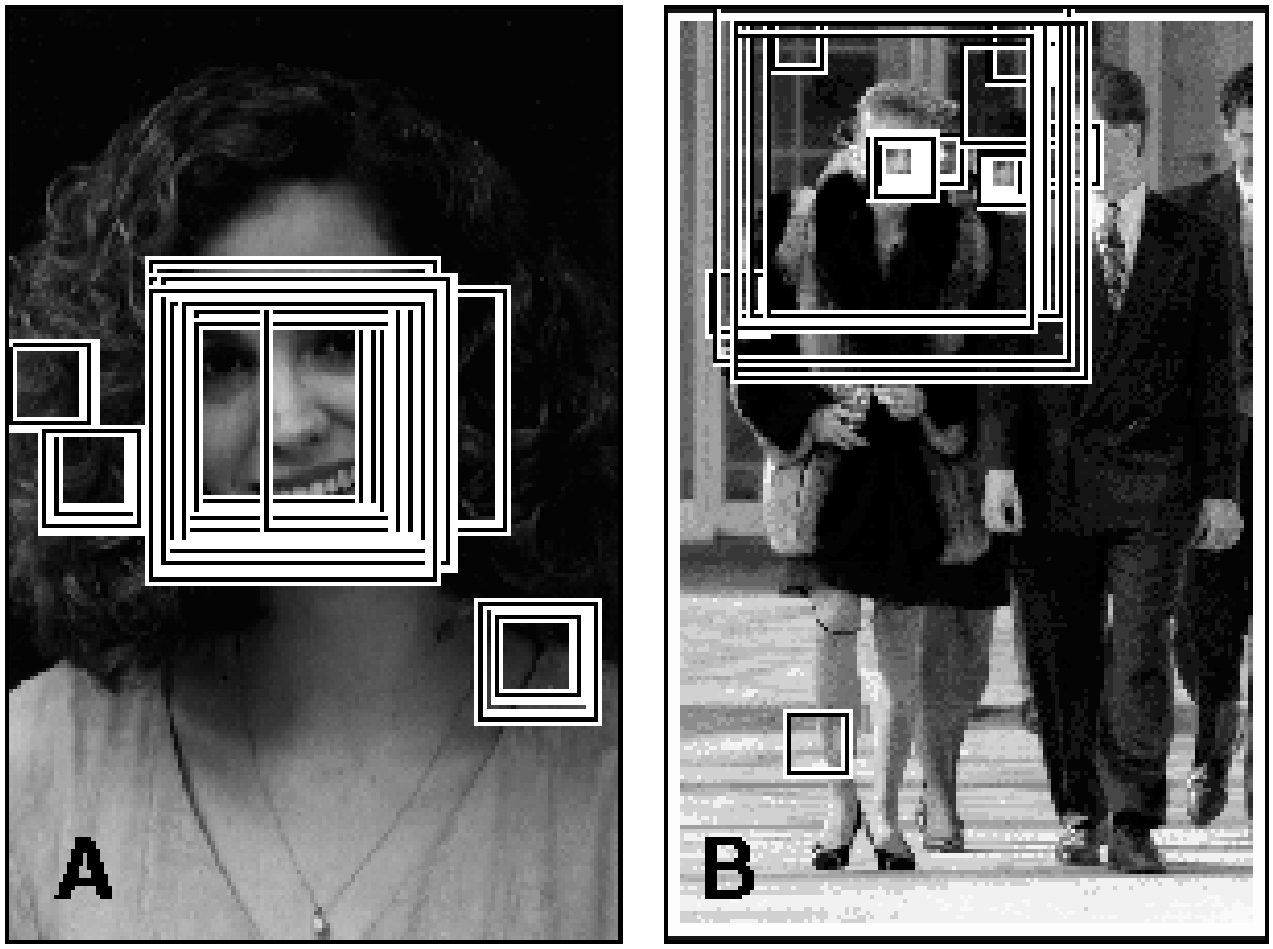
During training, the partially-trained system is applied to images of scenery which do not contain faces (like the one on the left). Any regions in the image detected as faces (which are expanded and shown on the right) are errors, which can be added into the set of negative training examples.



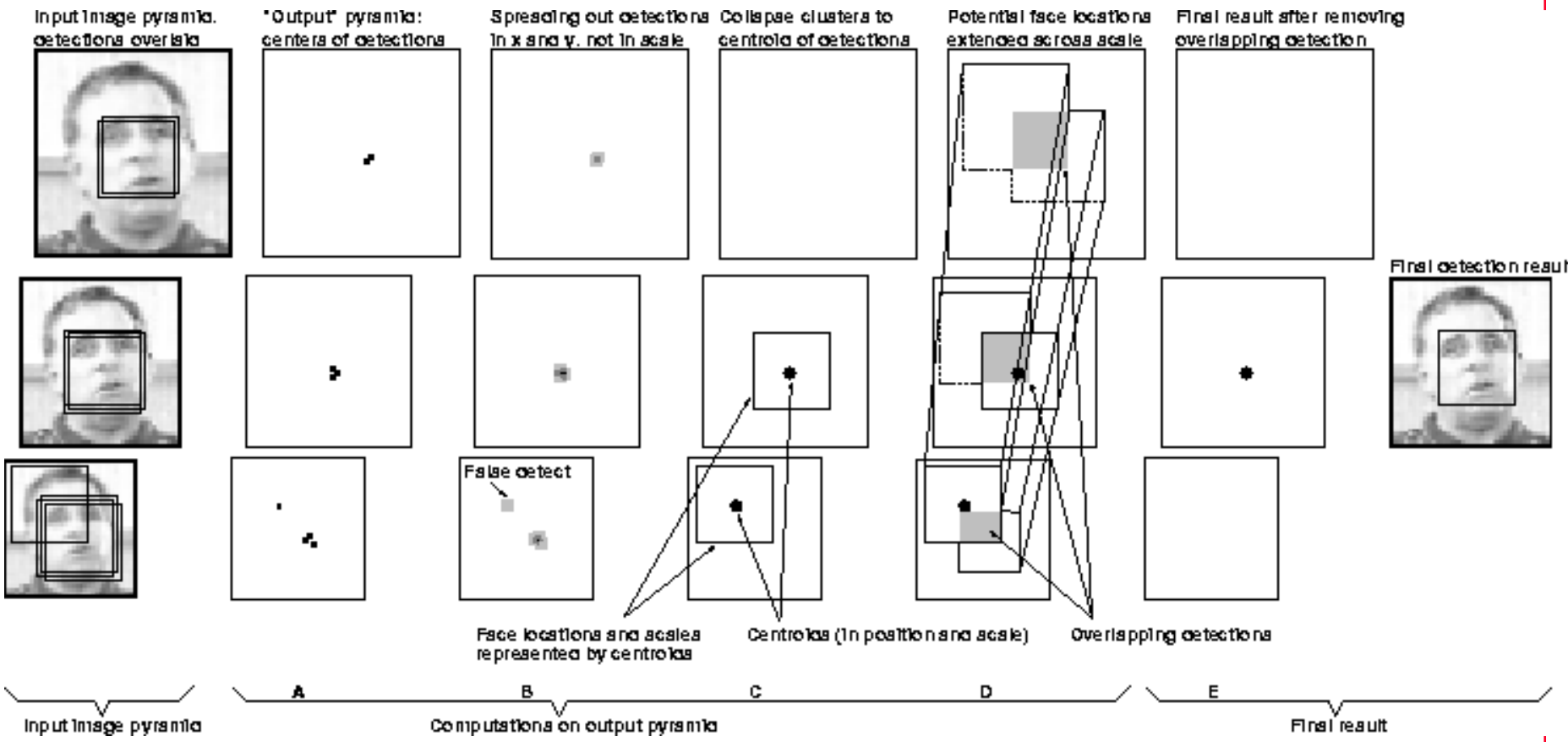


From: http://www.ius.cs.cmu.edu/IUS/har2/har/www/CMU-CS-95-158R/

Images with all the above threshold detections indicated by boxes.



From: http://www.ius.cs.cmu.edu/IUS/har2/har/www/CMU-CS-95-158R/



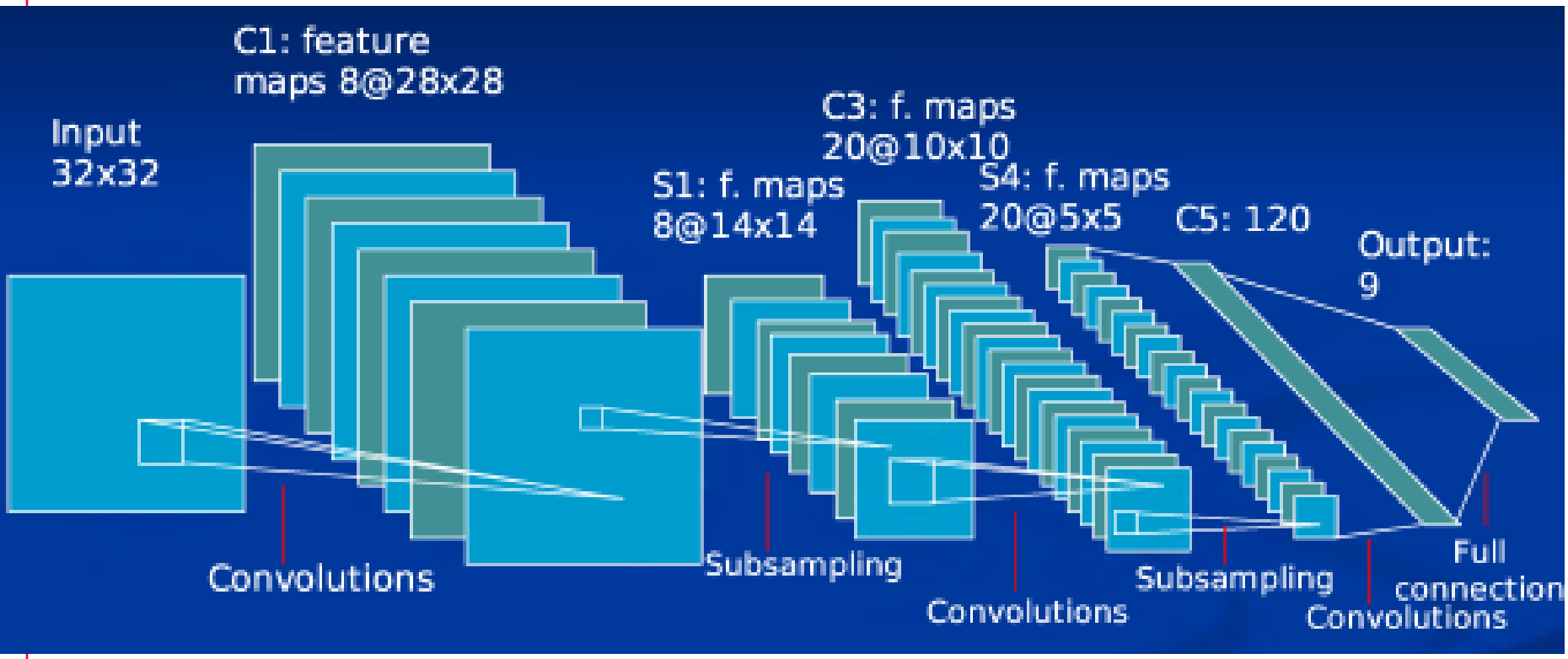
The framework used for merging multiple detections from a single network: A) The detections are recorded in an image pyramid. B) The detections are ``spread out'' and a threshold is applied. C) The centroids in scale and position are computed, and the regions contributing to each centroid are collapsed to single points. In the example shown, this leaves only two detections in the output pyramid. D) The final step is to check the proposed face locations for overlaps, and E) to remove overlapping detections if they exist. In this example, removing the overlapping detection eliminates what would otherwise be a false positive.

From: http://www.ius.cs.cmu.edu/IUS/har2/har/www/CMU-CS-95-158R/

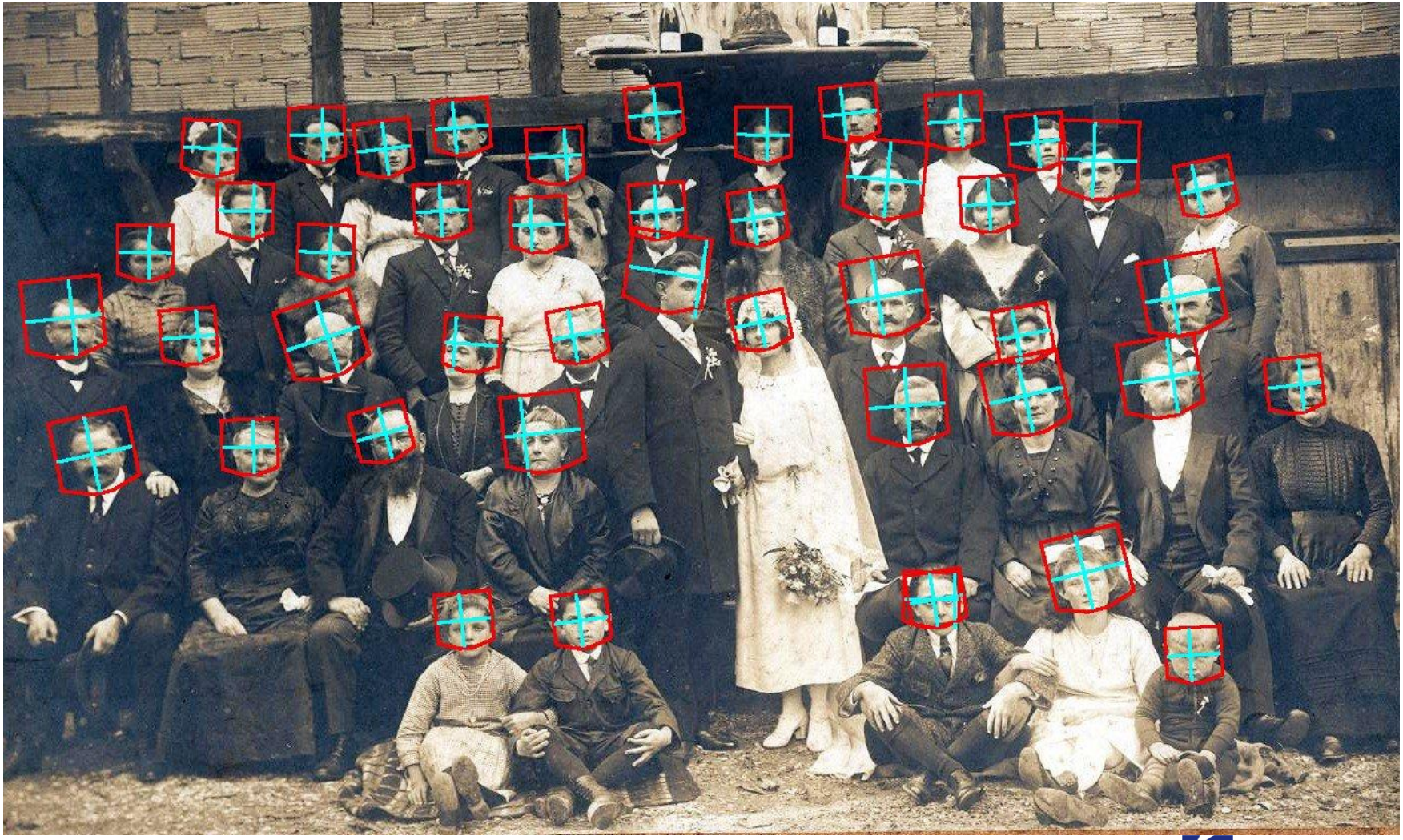

A: 57/57/3
B: 2/2/0
C: 1/1/0
D: 9/9/0
E: 15/15/0
F: 11/11/0
G: 2/1/0
H: 3/3/0
M: 1/1/0
I: 7/5/0
J: 8/7/1
K: 14/14/0
L: 1/1/0

Osadchy, Miller, LeCun.
Face Detection and Pose Estimation, 2004

- Application of Convolutional Neural Networks to Face Detection



Osadchy, Miller, LeCun.
Face Detection and Pose Estimation, 2004



Overview

- Specific Object Recognition
  - Lowe‘04
  - Rothganger et al.‘03
- Face Dectection
  - Sub-space methods
  - Neural Network-based approaches
  - Viola & Jones
- Face Recognition

Rapid Object Detection Using a Boosted Cascade of Simple Features

Paul Viola Michael J. Jones
Mitsubishi Electric Research Laboratories (MERL)
Cambridge, MA

Most of this work was done at Compaq CRL before the authors moved to MERL

The Viola/Jones Face Detector

- A seminal approach to real-time object detection
- Training is slow, but detection is very fast
- Key ideas

  Integral images for fast feature evaluation

  Boosting for feature selection

  Attentional cascade for fast rejection of non-face windows

P. Viola and M. Jones. *Rapid object detection using a boosted cascade of simple features.* CVPR 2001.

P. Viola and M. Jones. *Robust real-time face detection.* IJCV 57(2), 2004.

Image Features

"Rectangle filters"



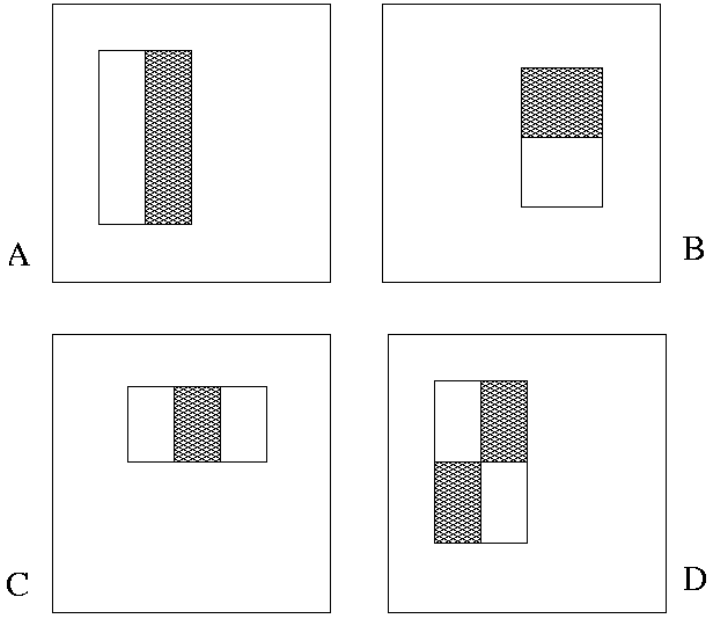
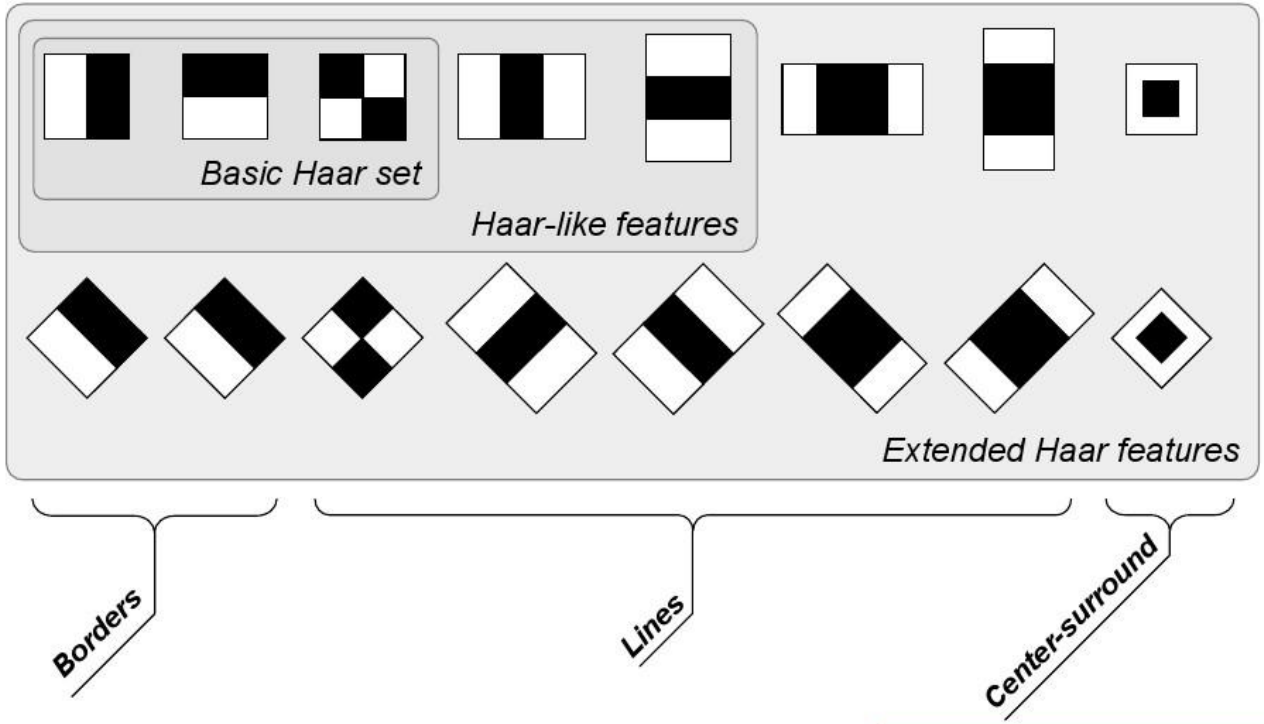


Image Features

Haar features

Different sets



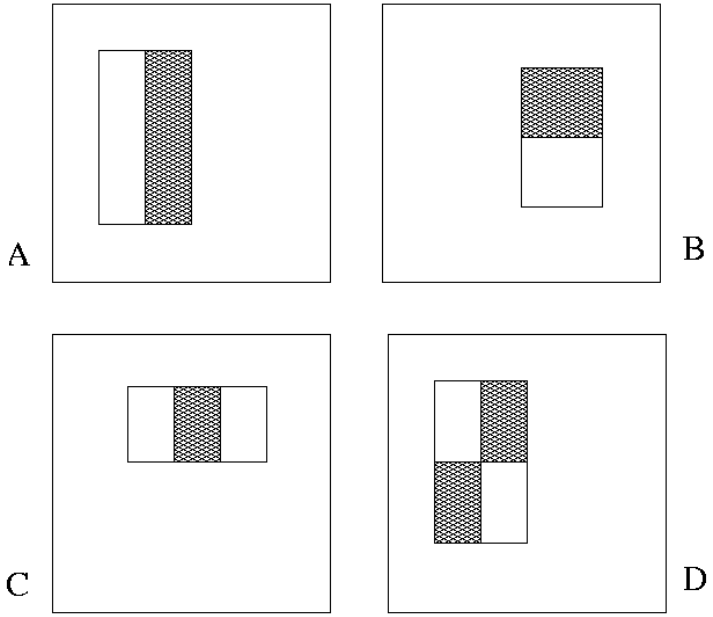
[Papageorgiou00, Viola01, Lienhart02]

Computer Vision Center
UAB Universitat Autònoma de Barcelona

Image Features

"Rectangle filters"

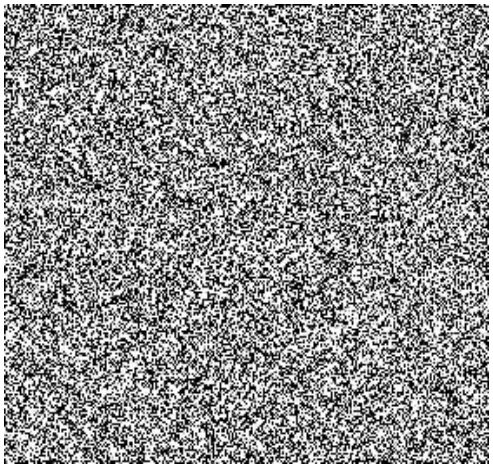




Value =

∑ (pixels in white area) –
∑ (pixels in black area)

Example

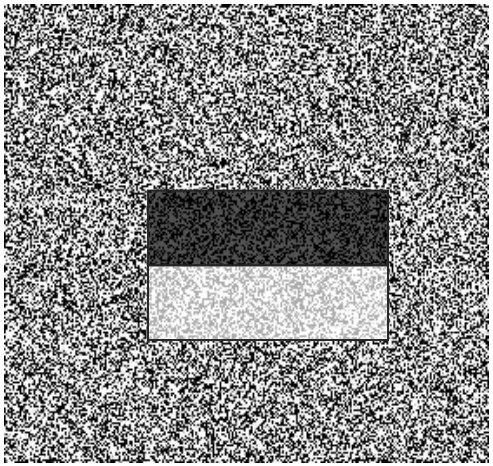


Source



Result





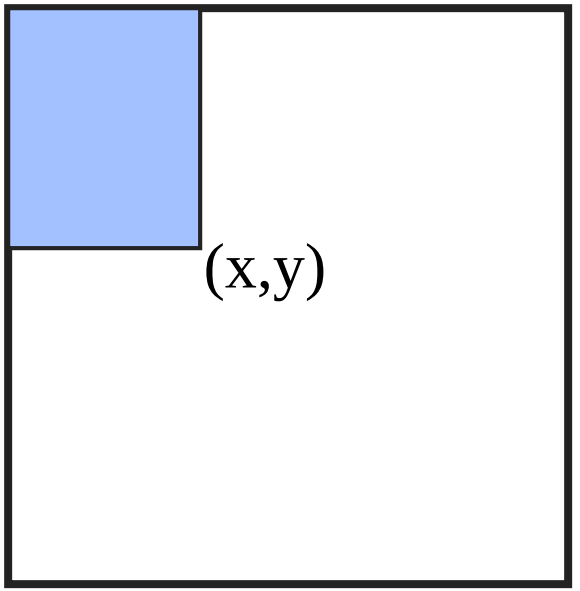




Slide: S. Lazebnik

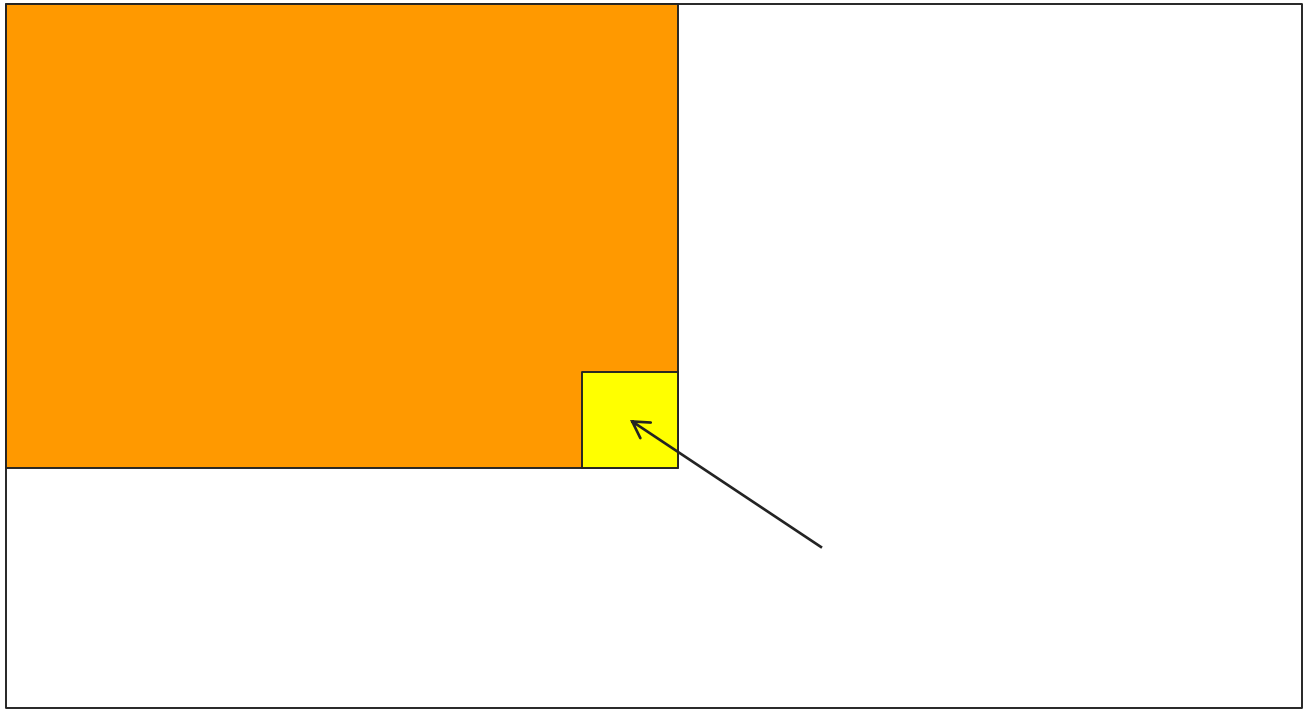
Fast computation with integral images

- The *integral image* computes a value at each pixel (*x*,*y*) that is the sum of the pixel values above and to the left of (*x*,*y*), inclusive
- This can quickly be computed in one pass through the image



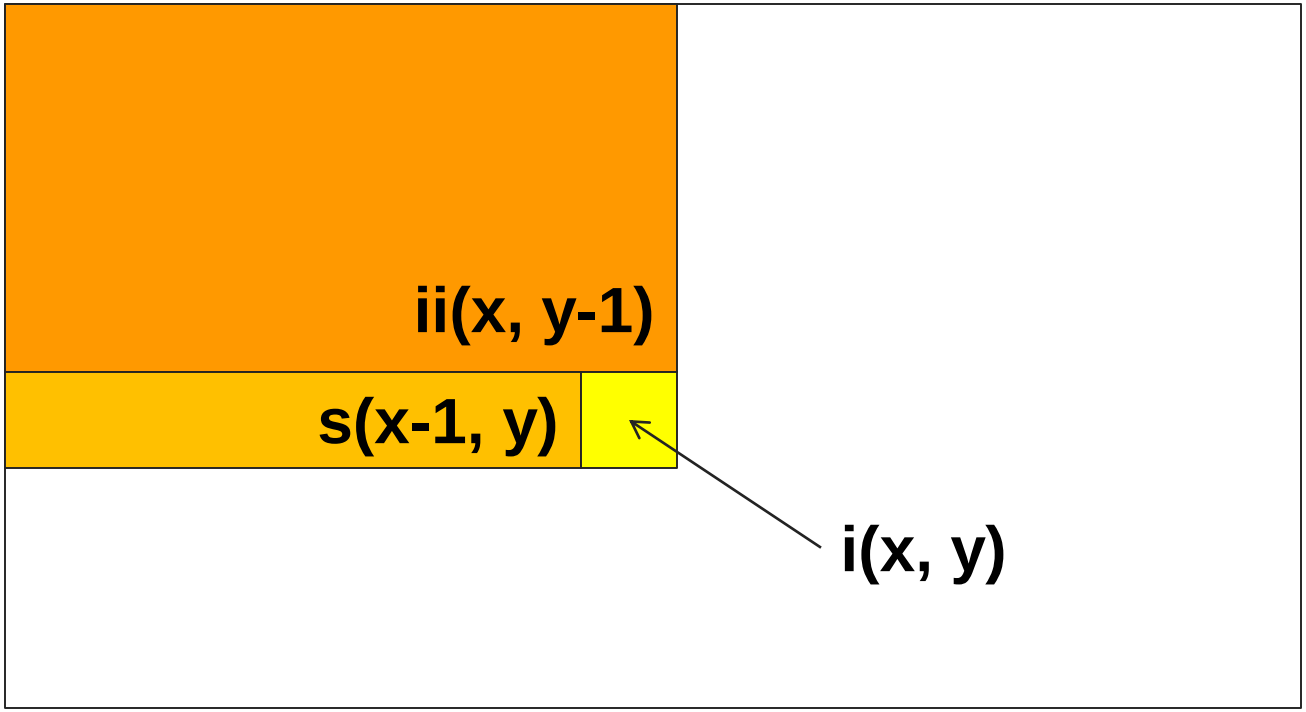
Slide: S. Lazebnik

Computing the integral image



Slide: S. Lazebnik

Computing the integral image



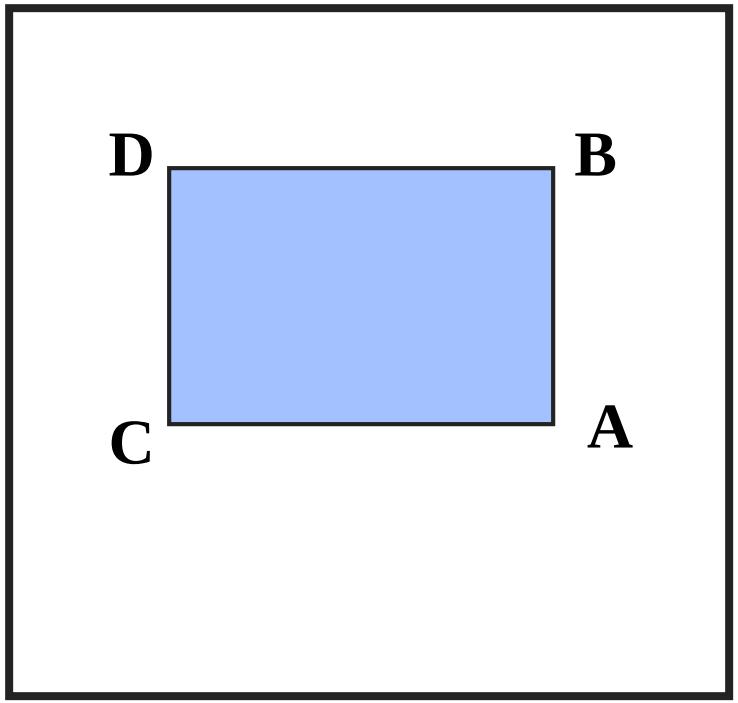
- Cumulative row sum: s(x, y) = s(x–1, y) + i(x, y)
- Integral image: ii(x, y) = ii(x, y−1) + s(x, y)

MATLAB: ii = cumsum(cumsum(double(i)), 2);

Computing sum within a rectangle

- Let A,B,C,D be the values of the integral image at the corners of a rectangle
- Then the sum of original image values within the rectangle can be computed as:

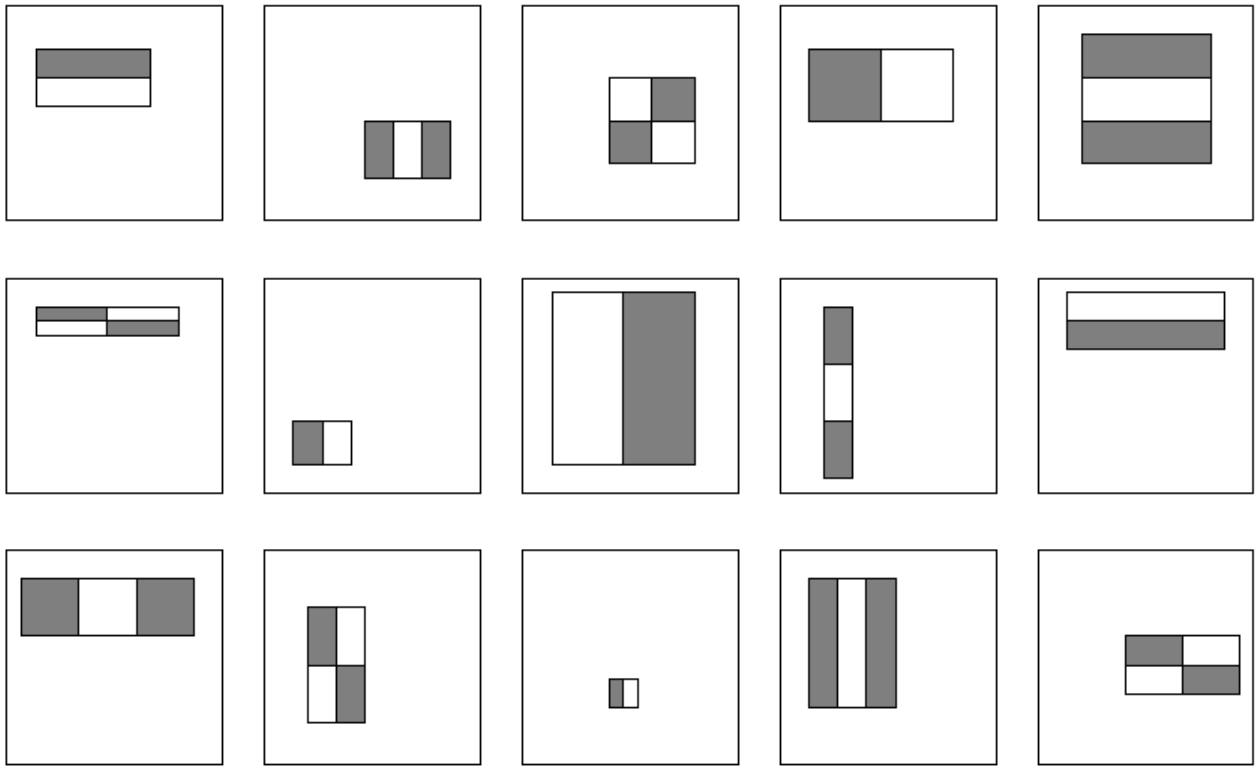
  sum = A – B – C + D
- Only 3 additions are required for any size of rectangle!



Slide: S. Lazebnik

Feature selection

- For a 24x24 detection region, the number of possible rectangle features is ~160,000!



Feature selection

- For a 24x24 detection region, the number of possible rectangle features is ~160,000!
- At test time, it is impractical to evaluate the entire feature set
- Can we create a good classifier using just a small subset of all possible features?
- How to select such a subset?

Boosting

- Boosting is a classification scheme that works by combining *weak learners* into a more accurate ensemble classifier
- Training consists of multiple *boosting rounds*

  During each boosting round, we select a weak learner that does well on examples that were hard for the previous weak learners

  "Hardness" is captured by weights attached to training examples

Y. Freund and R. Schapire, A short introduction to boosting, *Journal of Japanese Society for Artificial Intelligence*, 14(5):771-780, September, 1999.

Training procedure

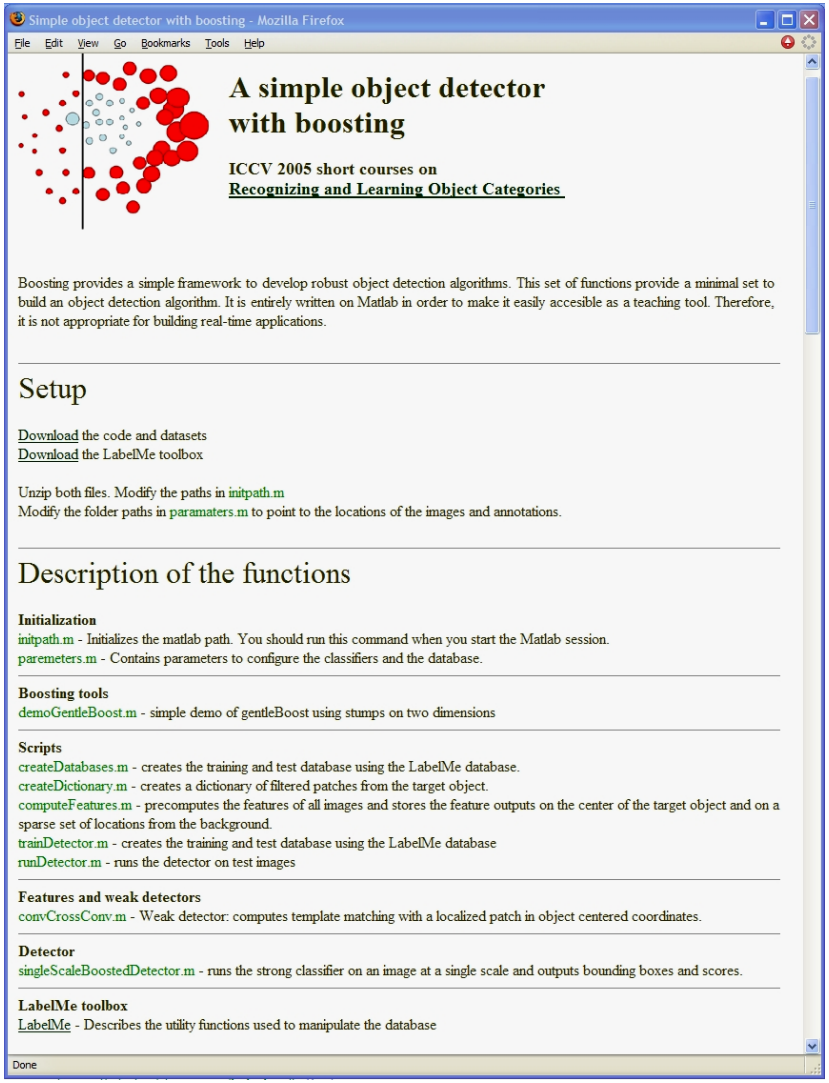
- Initially, weight each training example equally
- In each boosting round:

  Find the weak learner that achieves the lowest *weighted* training error

  Raise the weights of training examples misclassified by current weak learner
- Compute final classifier as linear combination of all weak learners (weight of each learner is directly proportional to its accuracy)
- Exact formulas for re-weighting and combining weak learners depend on the particular boosting scheme (e.g., AdaBoost)

Y. Freund and R. Schapire, A short introduction to boosting, *Journal of Japanese Society for Artificial Intelligence*, 14(5):771-780, September, 1999.

A simple object detector with Boosting



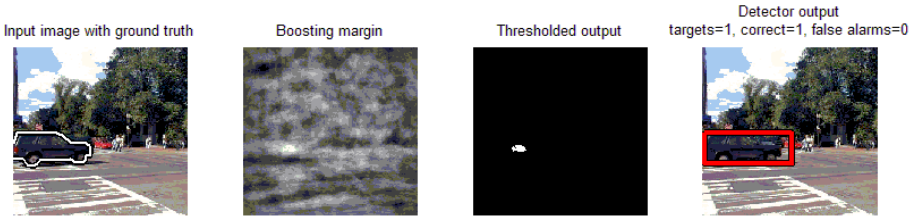
Download

- Toolbox for manipulating dataset
- Code and dataset

Matlab code

- Gentle boosting
- Object detector using a part based model

Dataset with cars and computer monitors



http://people.csail.mit.edu/torralba/iccv2005/

Why boosting?

- A simple algorithm for learning robust classifiers
  - Freund & Shapire, 1995
  - Friedman, Hastie, Tibshhirani, 1998
- Provides efficient algorithm for sparse visual feature selection
  - *Tieu & Viola, 2000*
  - *Viola & Jones, 2003*
- Easy to implement, not requires external optimization tools.

Boosting

- Defines a classifier using an additive model:

$$F(x) = \alpha_1 f_1(x) + \alpha_2 f_2(x) + \alpha_3 f_3(x) + \ldots$$

Strong classifier

Features vector

Weight

Weak classifier

Boosting

- Defines a classifier using an additive model:

$$F(x) = \alpha_1 f_1(x) + \alpha_2 f_2(x) + \alpha_3 f_3(x) + \ldots$$

Strong classifier

Features vector

Weight

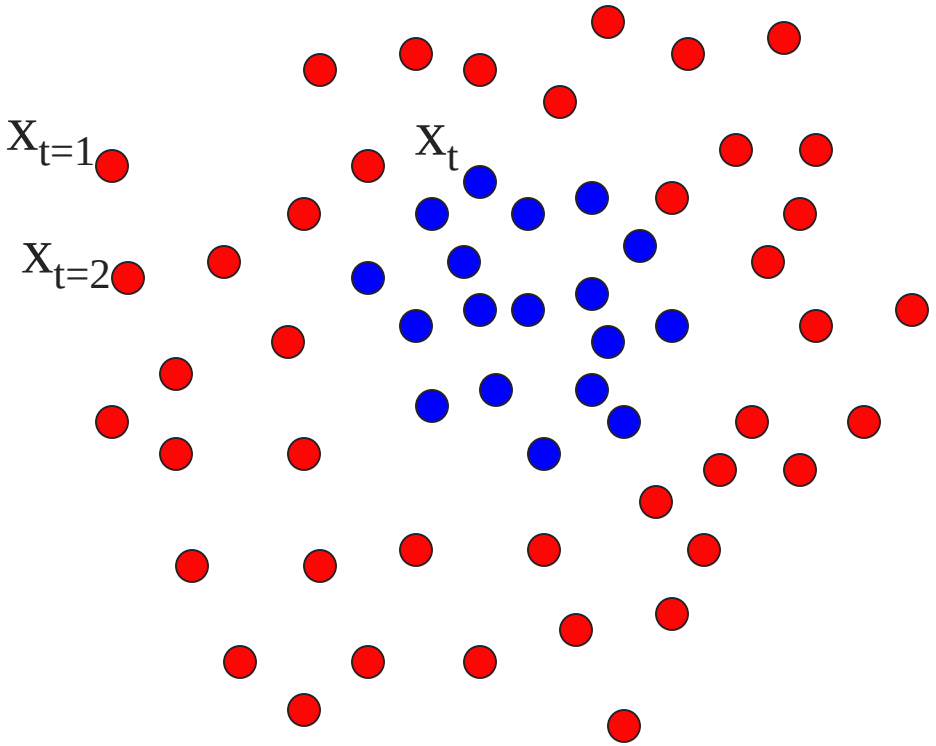
Weak classifier

- We need to define a family of weak classifiers

$f_k(x)$ from a family of weak classifiers

Boosting

- It is a sequential procedure:



Each data point has

a class label:

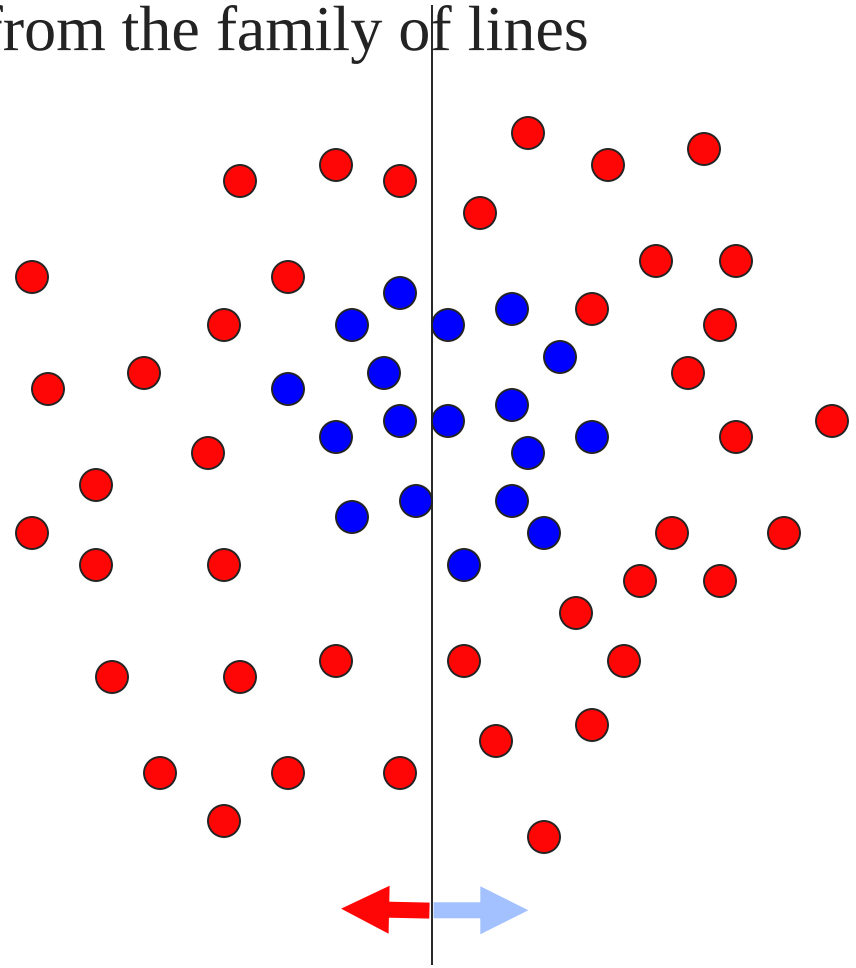
$$y_t = \begin{cases} +1 \ (\bullet) \\ -1 \ (\bullet) \end{cases}$$

and a weight:

$$w_t = 1$$

Toy example

Weak learners from the family of lines



Each data point has

a class label:

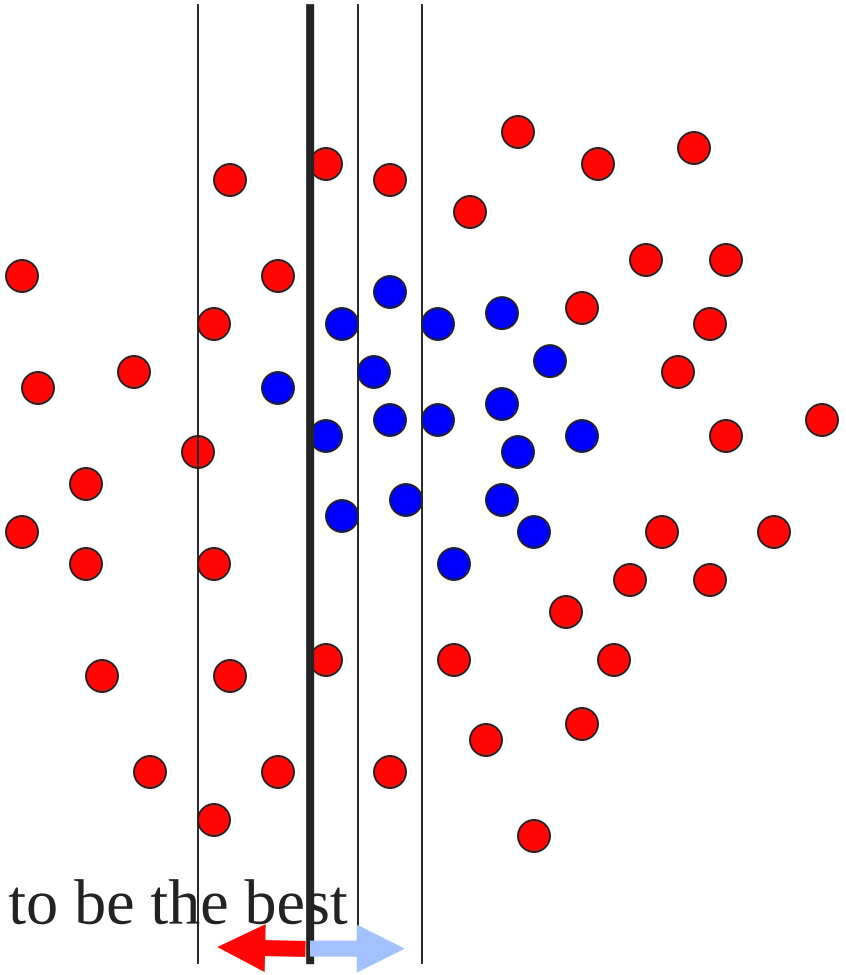
$$y_t = \begin{cases} +1 & (\bullet) \\ -1 & (\bullet) \end{cases}$$

and a weight:

$$w_t = 1$$

h => p(error) = 0.5 it is at chance

Toy example



Each data point has

a class label:

$$y_t = \begin{cases} +1 & (\bullet) \\ -1 & (\bullet) \end{cases}$$

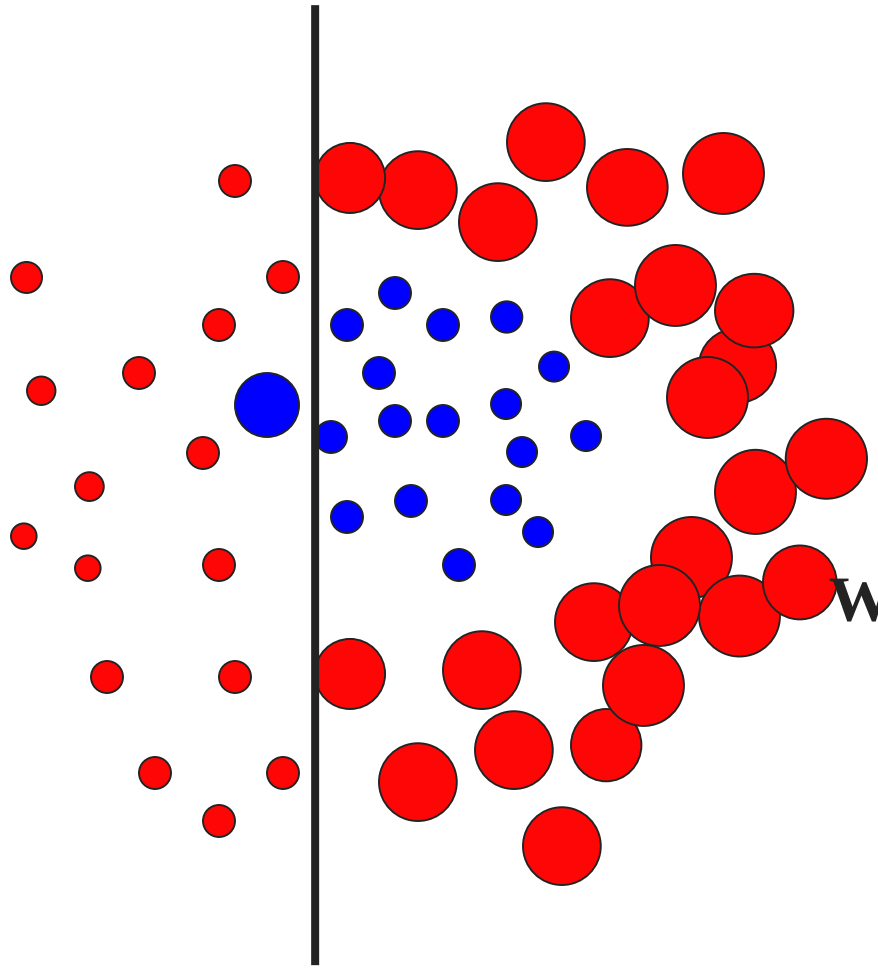
and a weight:

$$w_t = 1$$

This one seems to be the best

This is a '**weak classifier**': It performs slightly better than chance.

Toy example



Each data point has

a class label:

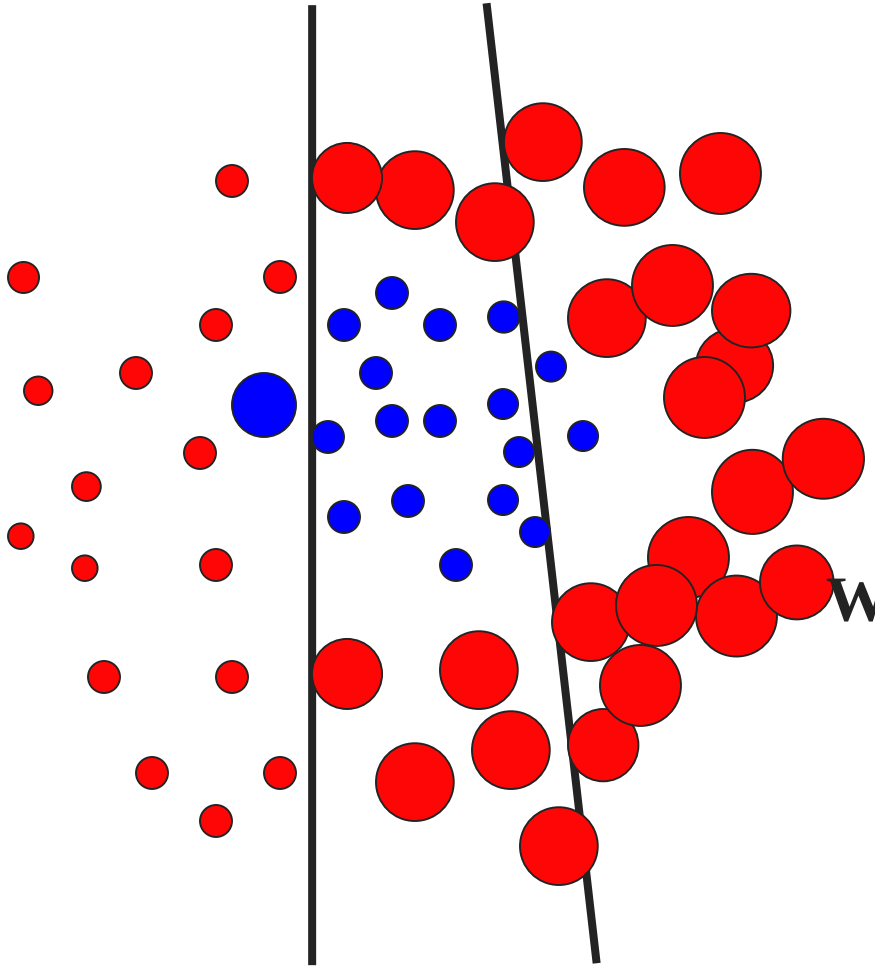
$$y_t = \begin{cases} +1 & (\bullet) \\ -1 & (\bullet) \end{cases}$$

We update the weights:

$$w_t \leftarrow w_t \exp\{-y_t H_t\}$$

We set a new problem for which the previous weak classifier performs at chance again

Toy example



Each data point has

a class label:

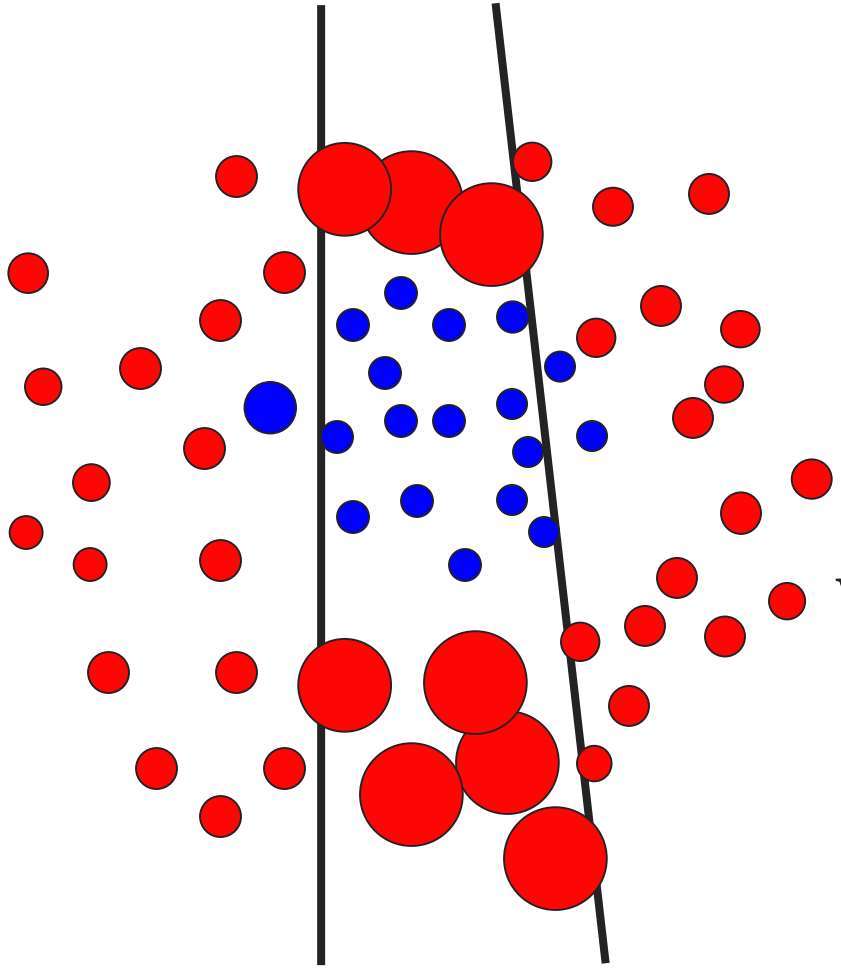
$$y_t = \begin{cases} +1 & (\bullet) \\ -1 & (\bullet) \end{cases}$$

We update the weights:

$$w_t \leftarrow w_t \exp\{-y_t H_t\}$$

We set a new problem for which the previous weak classifier performs at chance again

Toy example



Each data point has

a class label:

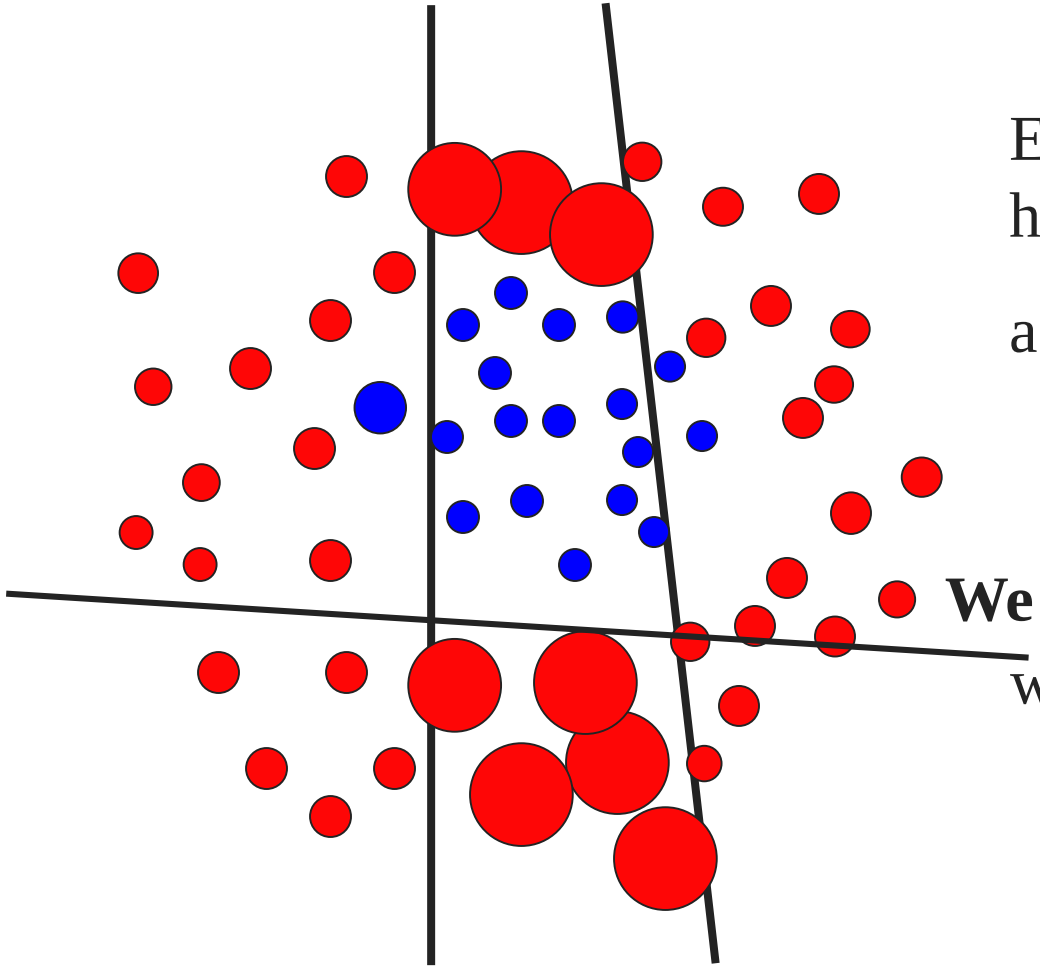
$$y_t = \begin{cases} +1 & (\bullet) \\ -1 & (\bullet) \end{cases}$$

We update the weights:

$$w_t \leftarrow w_t \exp\{-y_t H_t\}$$

We set a new problem for which the previous weak classifier performs at chance again

Toy example



Each data point has

a class label:

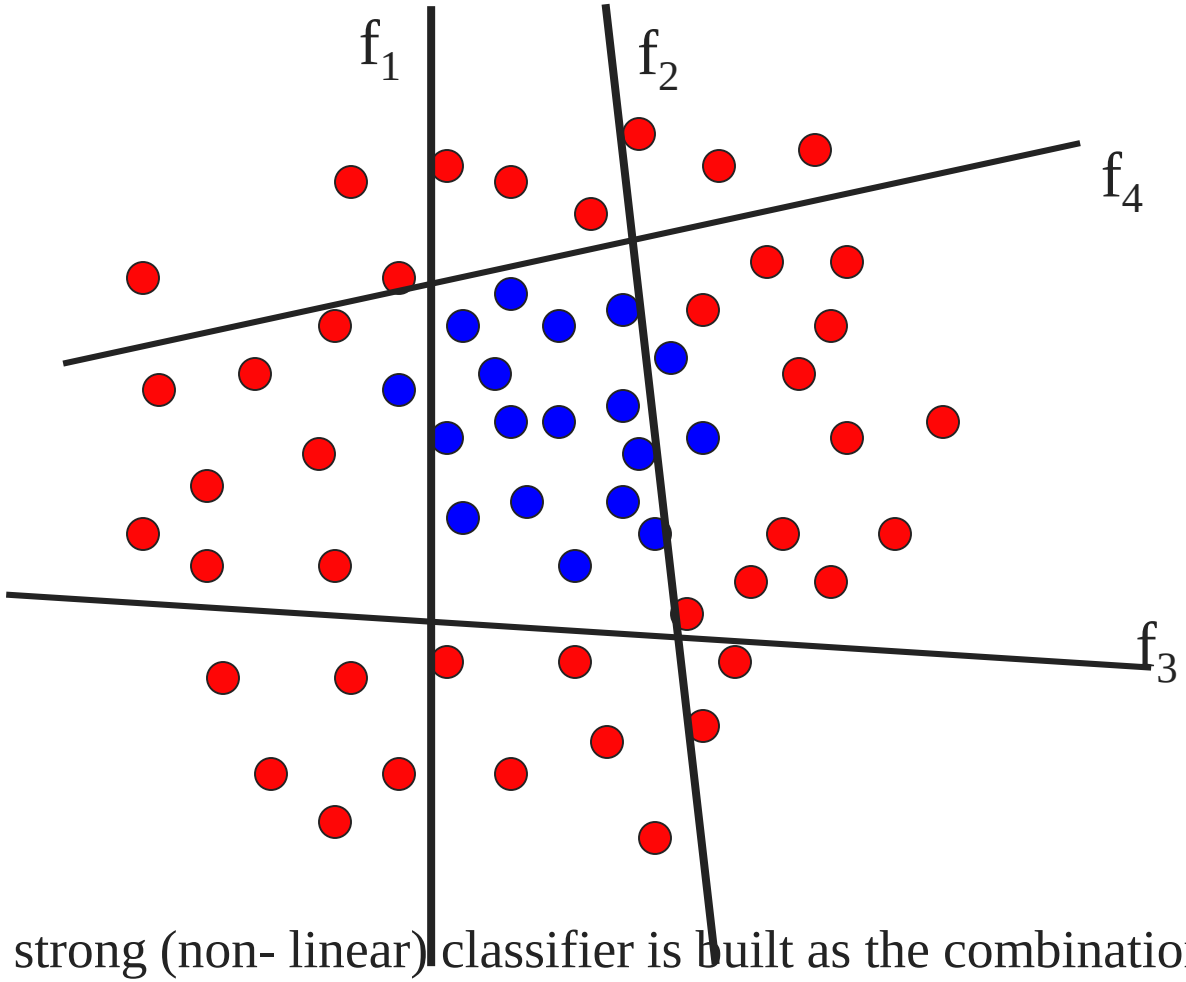
$$y_t = \begin{cases} +1 & (\bullet) \\ -1 & (\bullet) \end{cases}$$

We update the weights:

$$w_t \leftarrow w_t \exp\{-y_t H_t\}$$

We set a new problem for which the previous weak classifier performs at chance again

Toy example



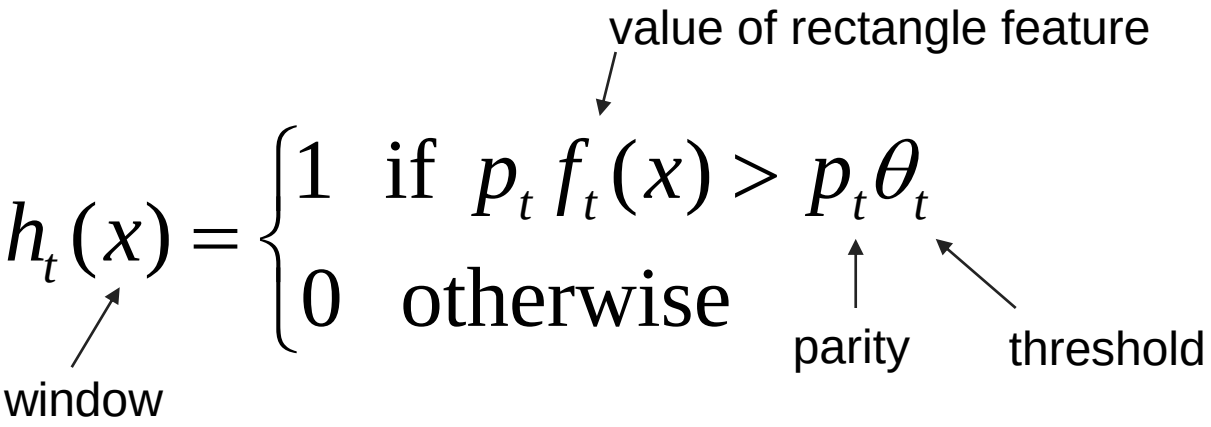
The strong (non- linear) classifier is built as the combination of all the weak (linear) classifiers.

Boosting vs. SVM

- Advantages of boosting
  - Integrates classifier training with feature selection
  - Complexity of training is linear instead of quadratic in the number of training examples
  - Flexibility in the choice of weak learners, boosting scheme
  - Testing is fast
  - Easy to implement
- Disadvantages
  - Needs many training examples
  - Training is slow
  - Often doesn't work as well as SVM (especially for many-class problems)

Boosting for face detection

- Define weak learners based on rectangle features

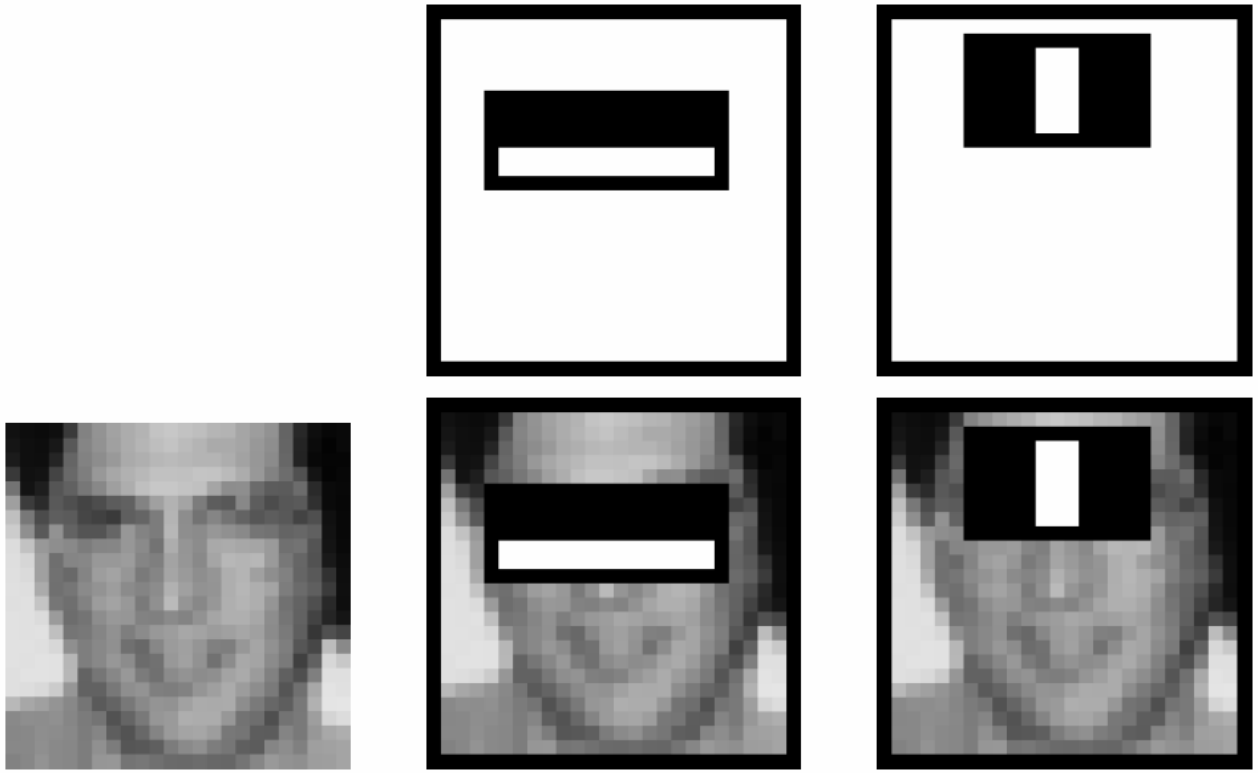


Boosting for face detection

- Define weak learners based on rectangle features
- For each round of boosting:
  - Evaluate each rectangle filter on each example
  - Select best filter/threshold combination based on weighted training error
  - Reweight examples

Boosting for face detection

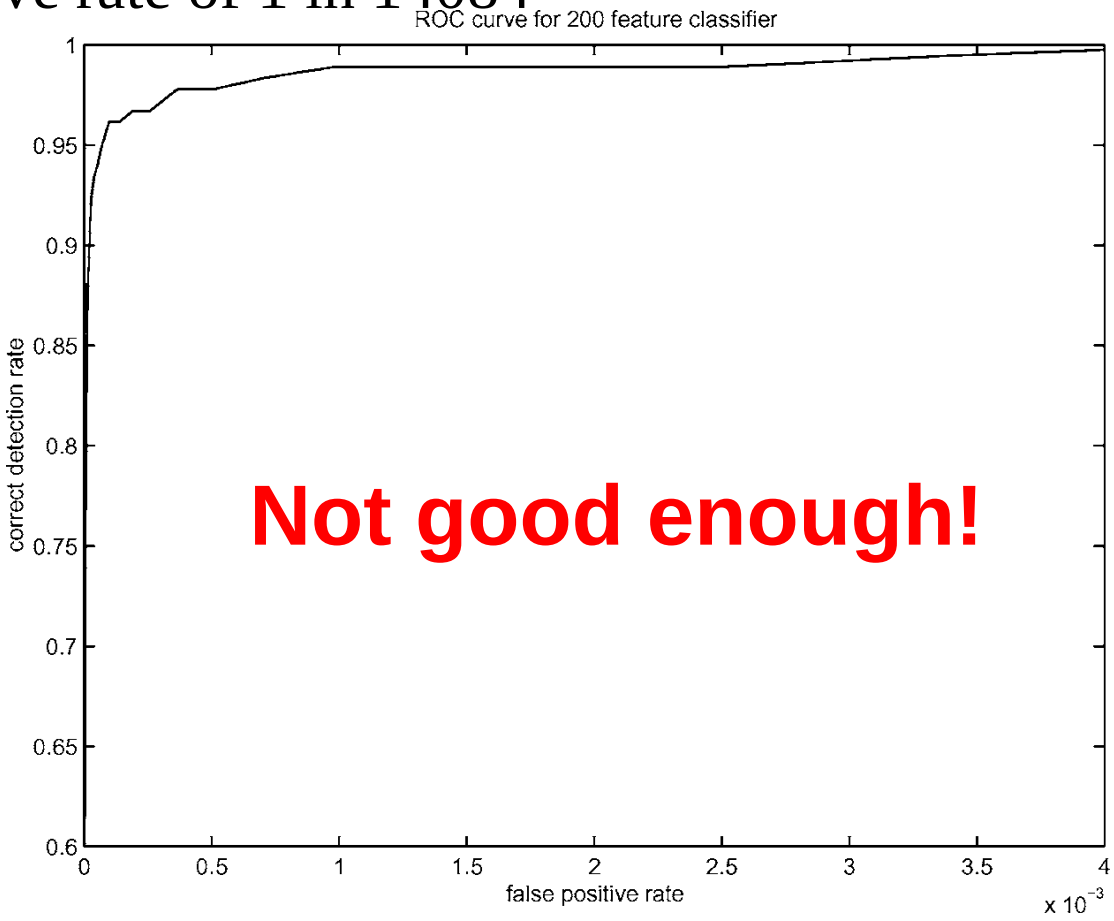
- First two features selected by boosting:
-



This feature combination can yield 100% detection rate and 50% false positive rate

Boosting for face detection

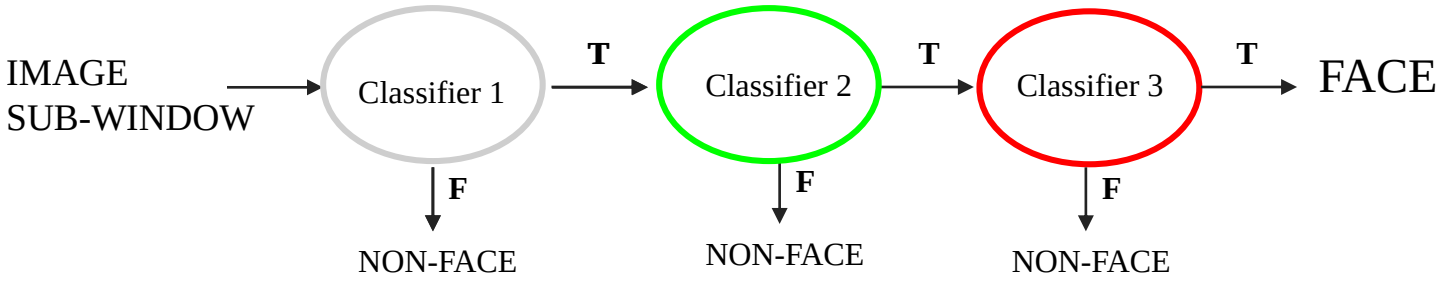
- A 200-feature classifier can yield 95% detection rate and a false positive rate of 1 in 14084



Receiver operating characteristic (ROC) curve

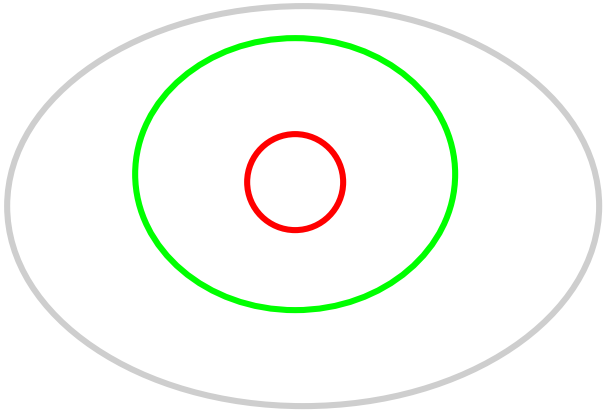
Attentional cascade

- We start with simple classifiers which reject many of the negative sub-windows while detecting almost all positive sub-windows
- Positive response from the first classifier triggers the evaluation of a second (more complex) classifier, and so on
- A negative outcome at any point leads to the immediate rejection of the sub-window

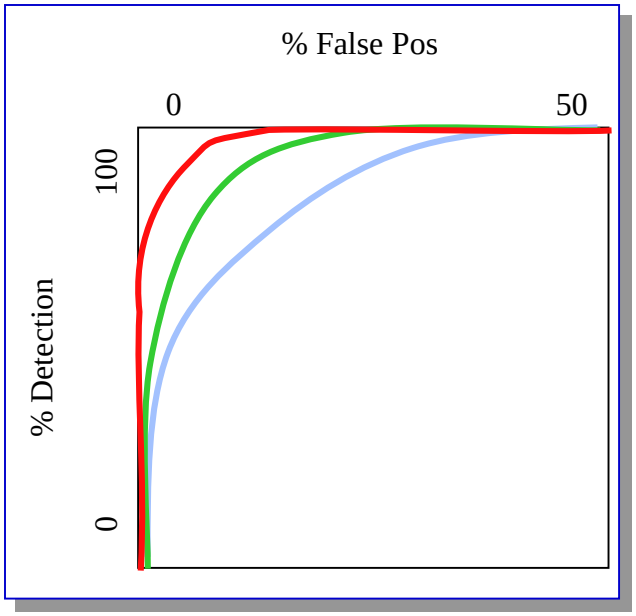


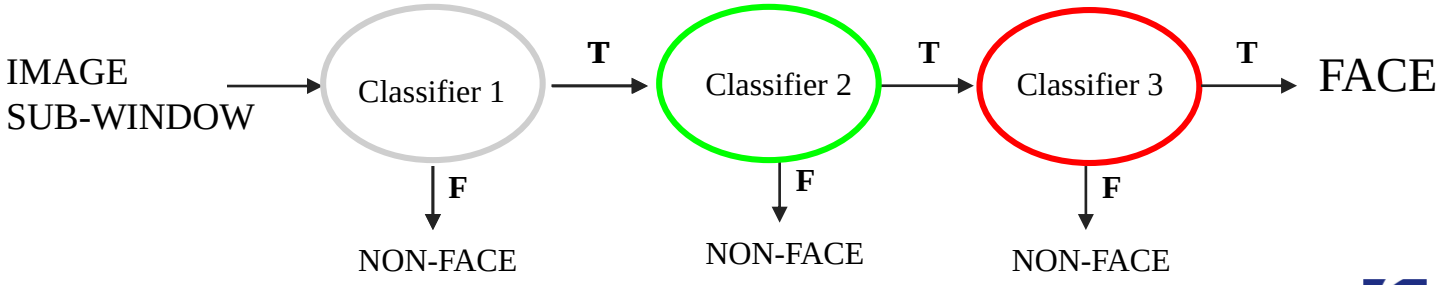
Attentional cascade

- Chain classifiers that are progressively more complex and have lower false positive rates:



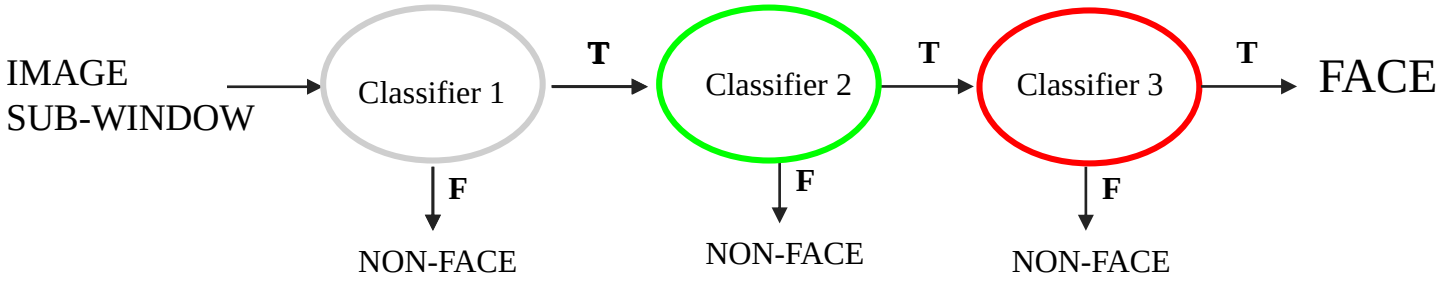
Receiver operating characteristic





Attentional cascade

- The detection rate and the false positive rate of the cascade are found by multiplying the respective rates of the individual stages
- A detection rate of 0.9 and a false positive rate on the order of 10^{-6} can be achieved by a
  10-stage cascade if each stage has a detection rate of 0.99 ($0.99^{10} \approx 0.9$) and a false positive rate of about 0.30 ($0.3^{10} \approx 6 \times 10^{-6}$)



Training the cascade

- Set target detection and false positive rates for each stage
- Keep adding features to the current stage until its target rates have been met

  Need to lower AdaBoost threshold to maximize detection (as opposed to minimizing total classification error)

  Test on a *validation set*
- If the overall false positive rate is not low enough, then add another stage
- Use false positives from current stage as the negative training examples for the next stage

The implemented system

- Training Data
  - 5000 faces
    - All frontal, rescaled to 24x24 pixels
  - 300 million non-faces
    - 9500 non-face images
  - Faces are normalized
    - Scale, translation
- Many variations
  - Across individuals
  - Illumination
  - Pose



System performance

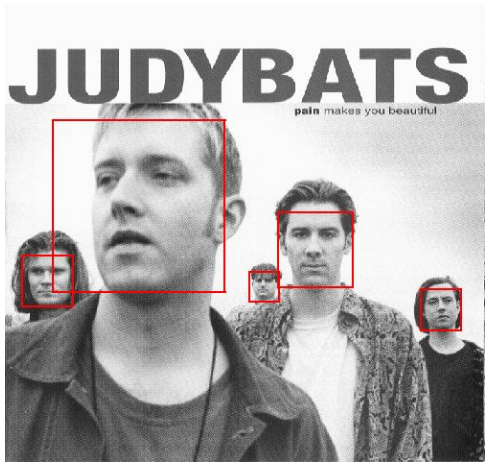
- Training time: “weeks” on 466 MHz Sun workstation
- 38 layers, total of 6061 features
- Average of 10 features evaluated per window on test set
- “On a 700 Mhz Pentium III processor, the face detector can process a 384 by 288 pixel image in about .067 seconds”

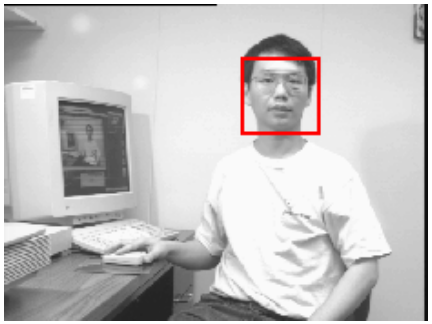
  15 Hz

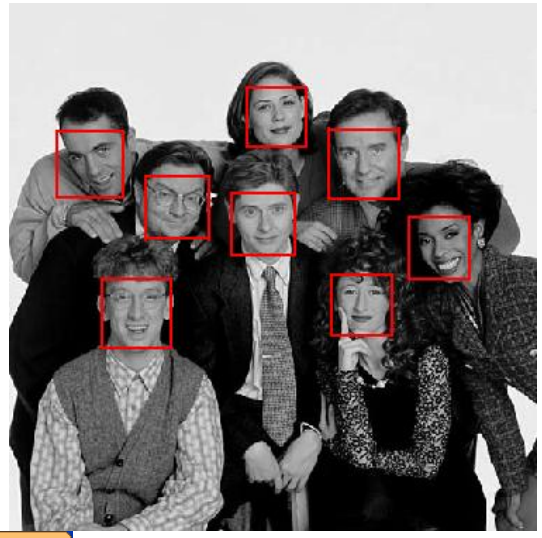
  15 times faster than previous detector of comparable accuracy (Rowley et al., 1998)

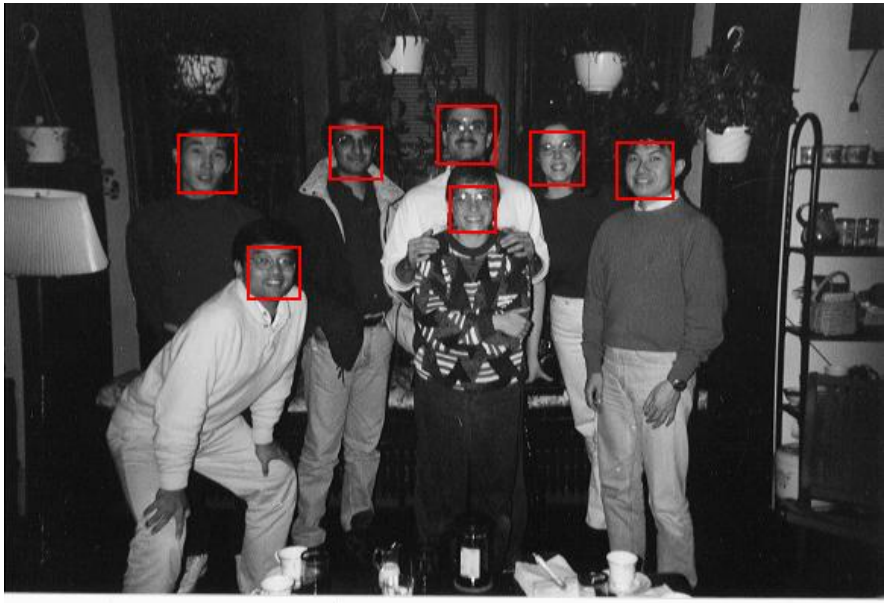
Output of Face Detector on Test Images





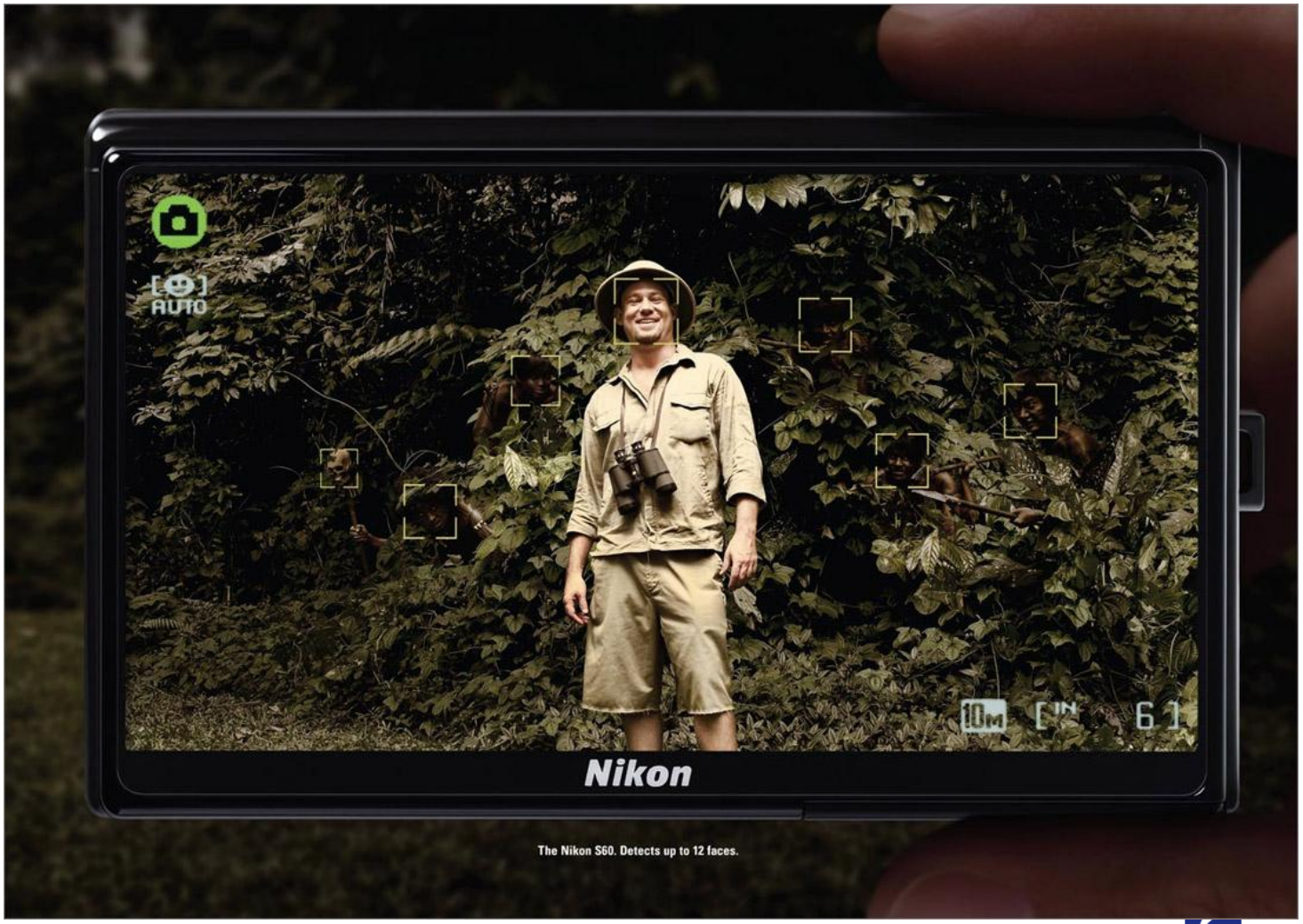






Funny Nikon ads

"The Nikon S60 detects up to 12 faces."



Slide: S. Lazebnik

Other detection tasks

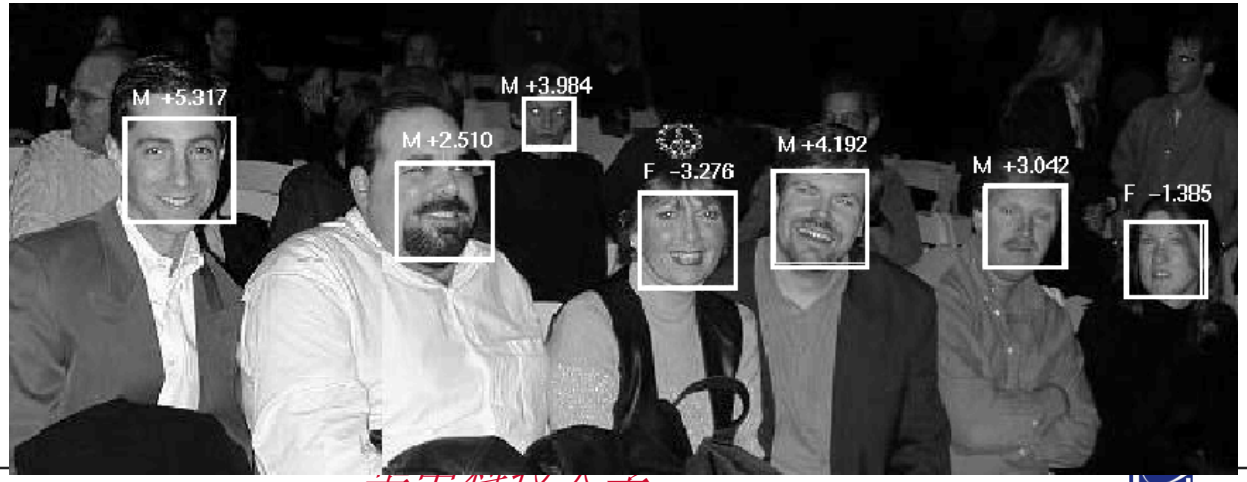


Facial Feature Localization



Profile Detection

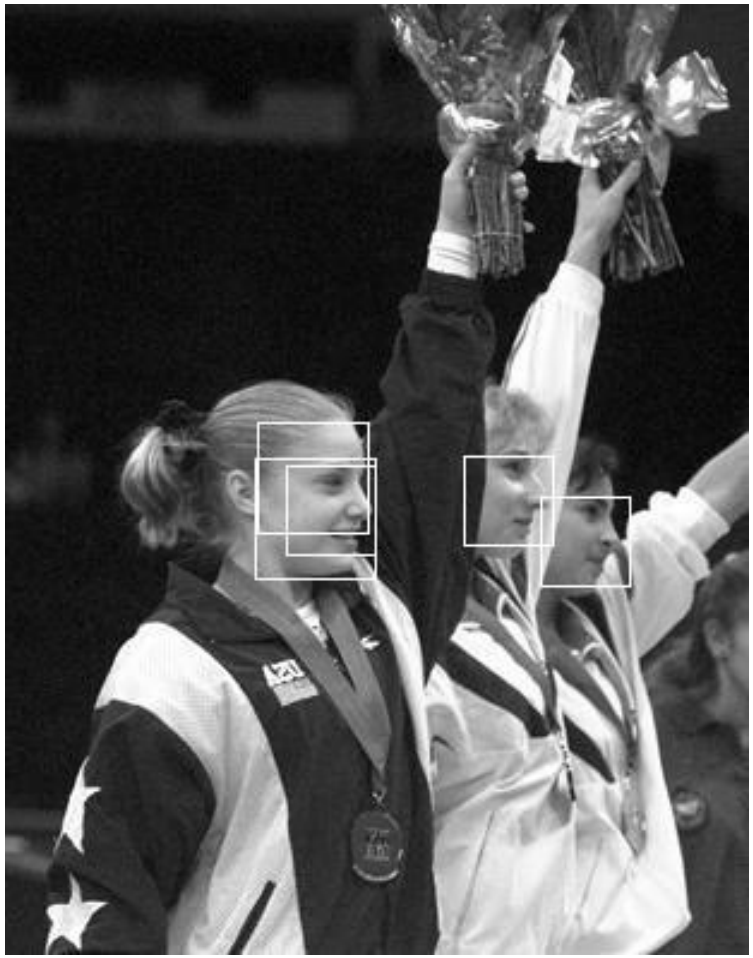
Male vs. female



Profile Detection

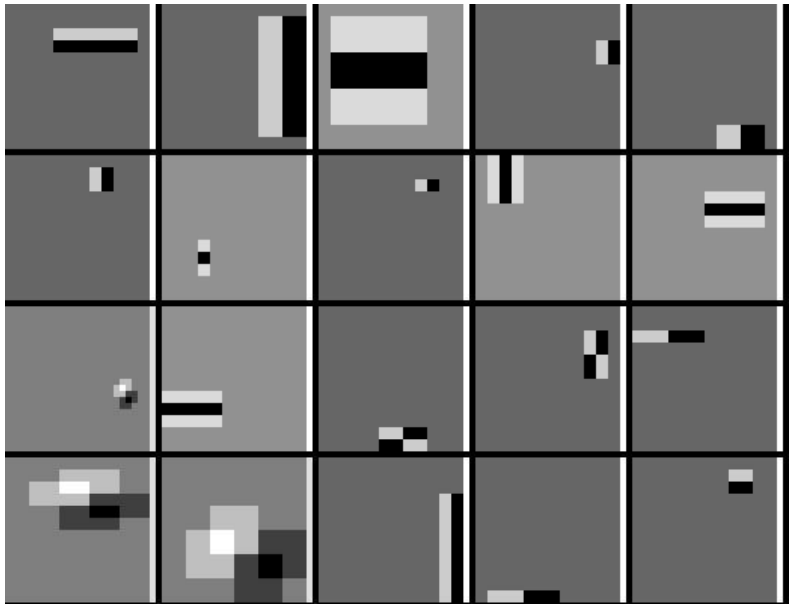






Profile Features





Summary: Viola/Jones detector

- Rectangle features
- Integral images for fast computation
- Boosting for feature selection
- Attentional cascade for fast rejection of negative windows

Overview

- Specific Object Recognition
  - Lowe‘04
  - Rothganger et al.‘03
- Face Dectection
  - Sub-space methods
  - Neural Network-based approaches
  - Viola & Jones
- Face Recognition

Subspace Methods

- PCA (“Eigenfaces”, Turk and Pentland)
- PCA (Bayesian, Moghaddam and Pentland)
- LDA/FLD (“Fisherfaces”, Belhumeur & Kreigman)
- ICA

Principal Component Analysis

- Given: N data points $x_1, \ldots, x_N$ in R^d
- We want to find a new set of features that are linear combinations of original ones:

$$u(x_i) = u^T(x_i - \mu)$$

$$\text{var}(x)=E(x-E(x))^2$$
$$=E(x^2) \text{ when } E(x)=0$$

(μ: mean of data points)

- What unit vector u in R^d captures the most variance of the data?

Principal Component Analysis

- Direction that maximizes the variance of the projected data:

$$var(u) = \frac{1}{N}\sum_{i=1}^{N} \underbrace{\mathbf{u}^{\mathrm{T}}(\mathbf{x}_i - \mu)}_{\text{Projection of data point}}(\mathbf{u}^{\mathrm{T}}(\mathbf{x}_i - \mu))^{\mathrm{T}}$$

$$= \mathbf{u}^{\mathrm{T}}\underbrace{\left[\sum_{i=1}^{N}(\mathbf{x}_i - \mu)(\mathbf{x}_i - \mu)^{\mathrm{T}}\right]}_{\text{Covariance matrix of data}}\mathbf{u}$$

$$= \mathbf{u}^{\mathrm{T}}\Sigma\mathbf{u}$$

The direction that maximizes the variance is the eigenvector associated with the largest eigenvalue of Σ

Eigenfaces: Key idea

- Assume that most face images lie on a low-dimensional subspace determined by the first k ($k<d$) directions of maximum variance
- Use PCA to determine the vectors $\mathbf{u}_1,\ldots\mathbf{u}_k$ that span that subspace: $\mathbf{x} \approx \boldsymbol{\mu} + w_1\mathbf{u}_1 + w_2\mathbf{u}_2 + \ldots + w_k\mathbf{u}_k$
- Represent each face using its "face space" coordinates $(w_1,\ldots w_k)$
- Perform nearest-neighbor recognition in "face space"

M. Turk and A. Pentland, Face Recognition using Eigenfaces, CVPR 1991

华中科技大学

Eigenfaces example

- Training images
- $\mathbf{x}_1,\ldots,\mathbf{x}_N$



Eigenfaces example

Top eigenvectors:

$\mathbf{u}_1,\ldots\mathbf{u}_k$



Mean: $\boldsymbol{\mu}$



Eigenfaces example

- Face **x** in "face space" coordinates:



$$\mathbf{x} \rightarrow [\mathbf{u}_1^{\mathrm{T}}(\mathbf{x}-\mu), \ldots, \mathbf{u}_k^{\mathrm{T}}(\mathbf{x}-\mu)]$$

$$= \quad w_1, \ldots, w_k$$

Eigenfaces example

- Face **x** in "face space" coordinates:



$$\mathbf{x} \rightarrow [\mathbf{u}_1^{\mathrm{T}}(\mathbf{x}-\mu), \ldots, \mathbf{u}_k^{\mathrm{T}}(\mathbf{x}-\mu)]$$

$$= \quad w_1, \ldots, w_k$$

- Reconstruction:



$$\hat{\mathbf{x}} = \boldsymbol{\mu} + w_1\mathbf{u}_1 + w_2\mathbf{u}_2 + w_3\mathbf{u}_3 + w_4\mathbf{u}_4 + \ldots$$

Summary: Recognition with eigenfaces

- Process labeled training images:
- Find mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$
- Find k principal components (eigenvectors of $\boldsymbol{\Sigma}$) $\mathbf{u}_1,\ldots\mathbf{u}_k$
- Project each training image $\mathbf{x}_i$ onto subspace spanned by principal components:
  $(w_{i1},\ldots,w_{ik}) = (\mathbf{u}_1^T(\mathbf{x}_i - \boldsymbol{\mu}), \ldots, \mathbf{u}_k^T(\mathbf{x}_i - \boldsymbol{\mu}))$

- Given novel image $\mathbf{x}$:
- Project onto subspace:
  $(w_1,\ldots,w_k) = (\mathbf{u}_1^T(\mathbf{x} - \boldsymbol{\mu}), \ldots, \mathbf{u}_k^T(\mathbf{x} - \boldsymbol{\mu}))$
- **Optional: check reconstruction error $\hat{x} - x$ to determine whether image is really a face**
- Classify as closest training face in k-dimensional subspace

Limitations

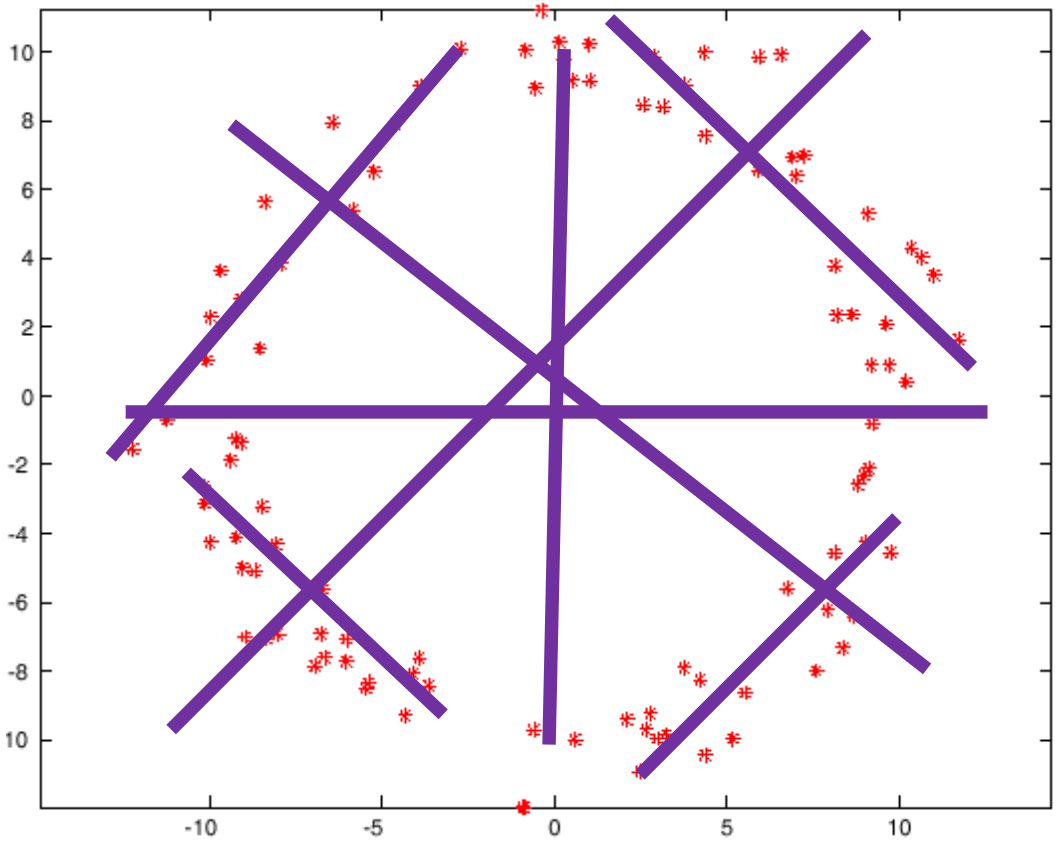
- Global appearance method: not robust to misalignment, background variation





Limitations

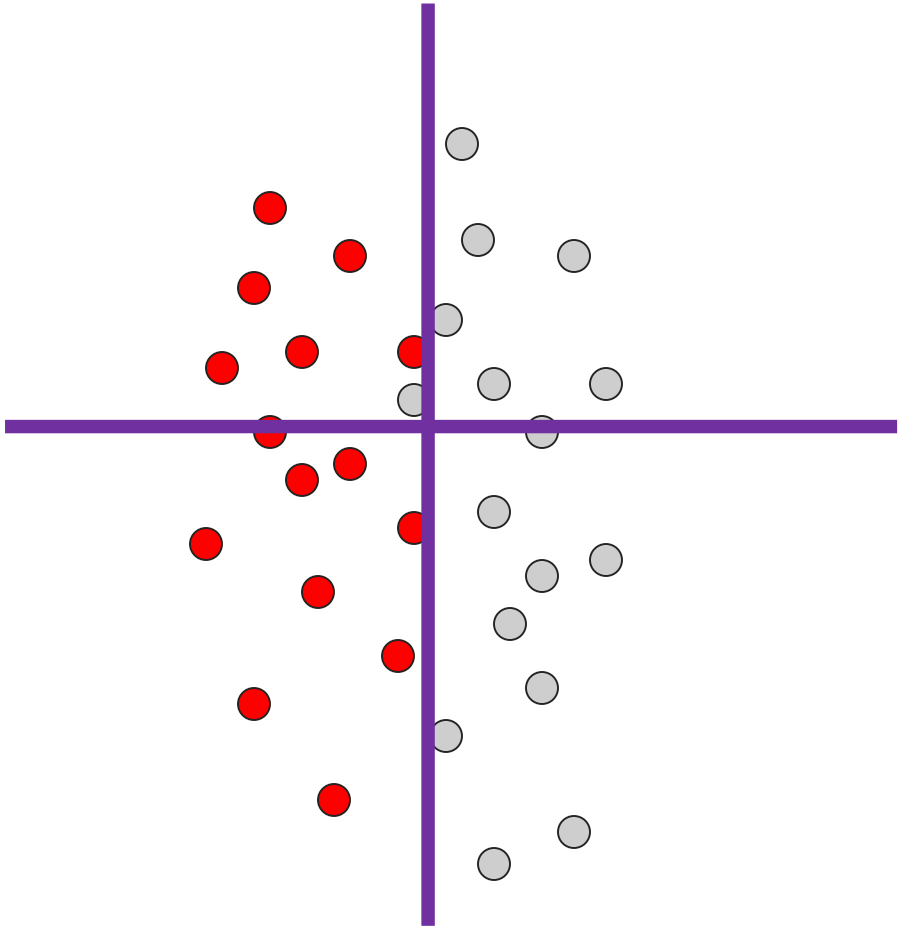
- PCA assumes that the data has a Gaussian distribution (mean μ, covariance matrix Σ)



The shape of this dataset is not well described by its principal components

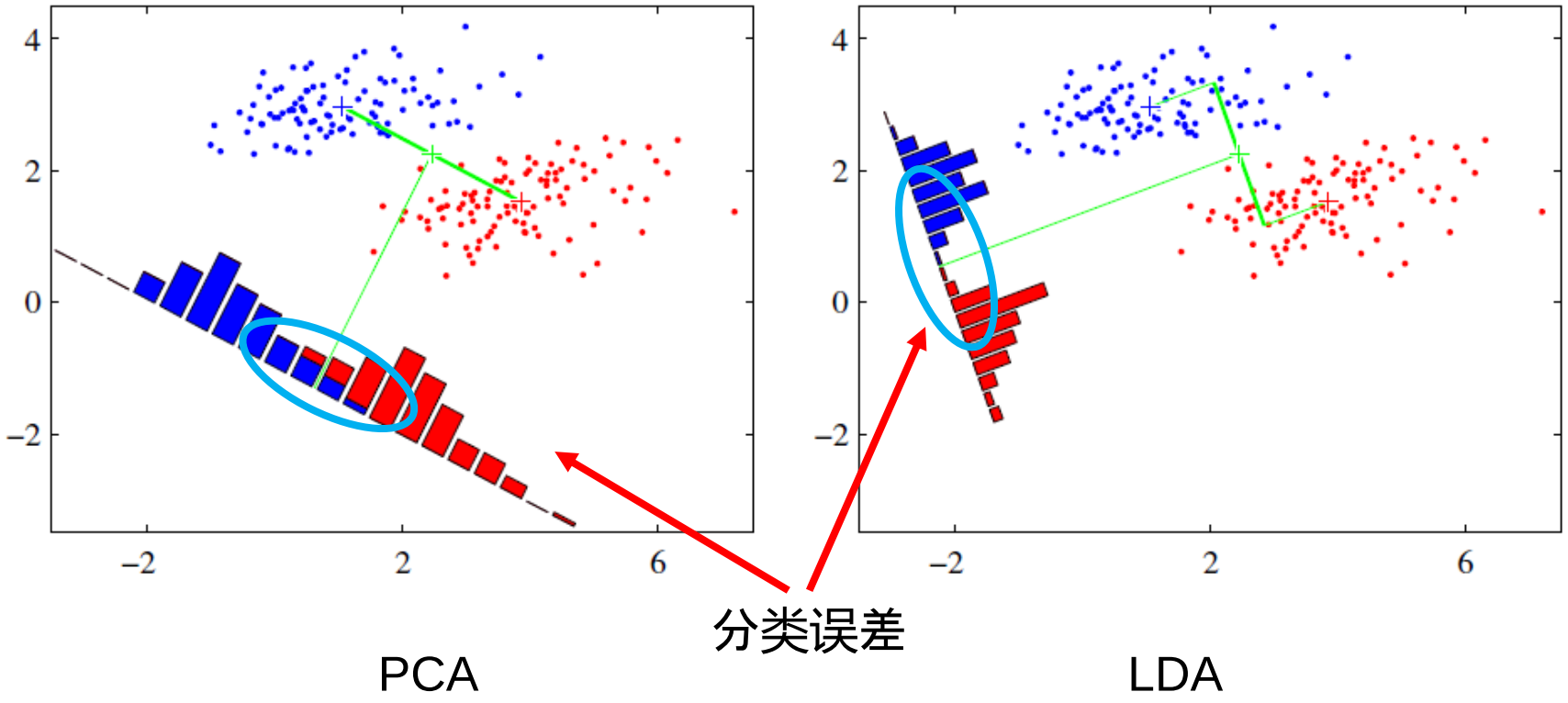
Limitations

- The direction of maximum variance is not always good for classification

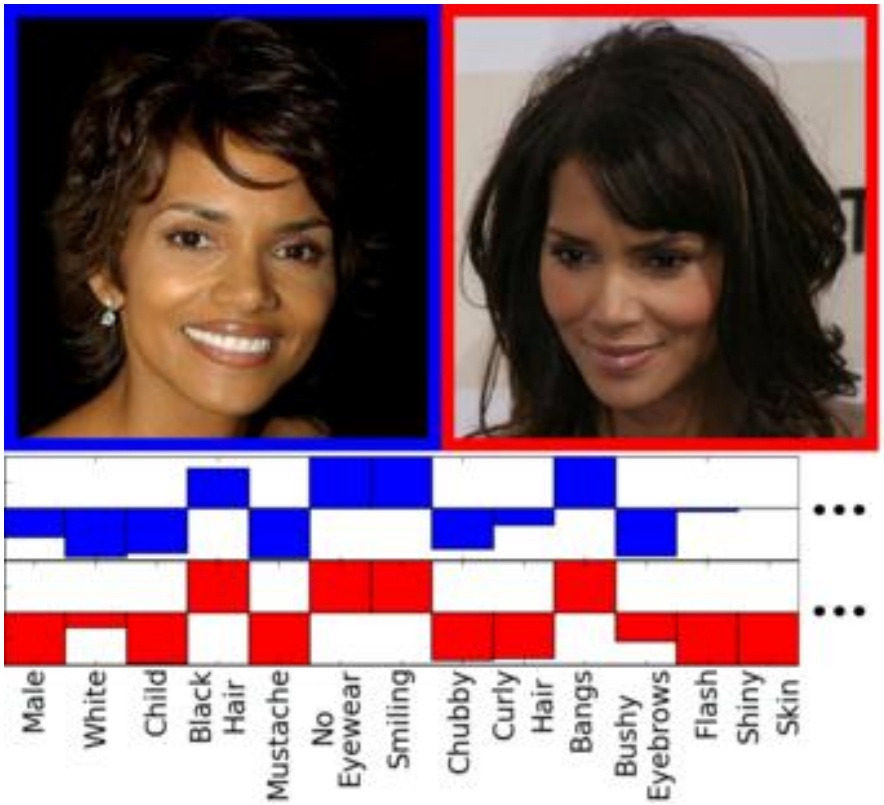


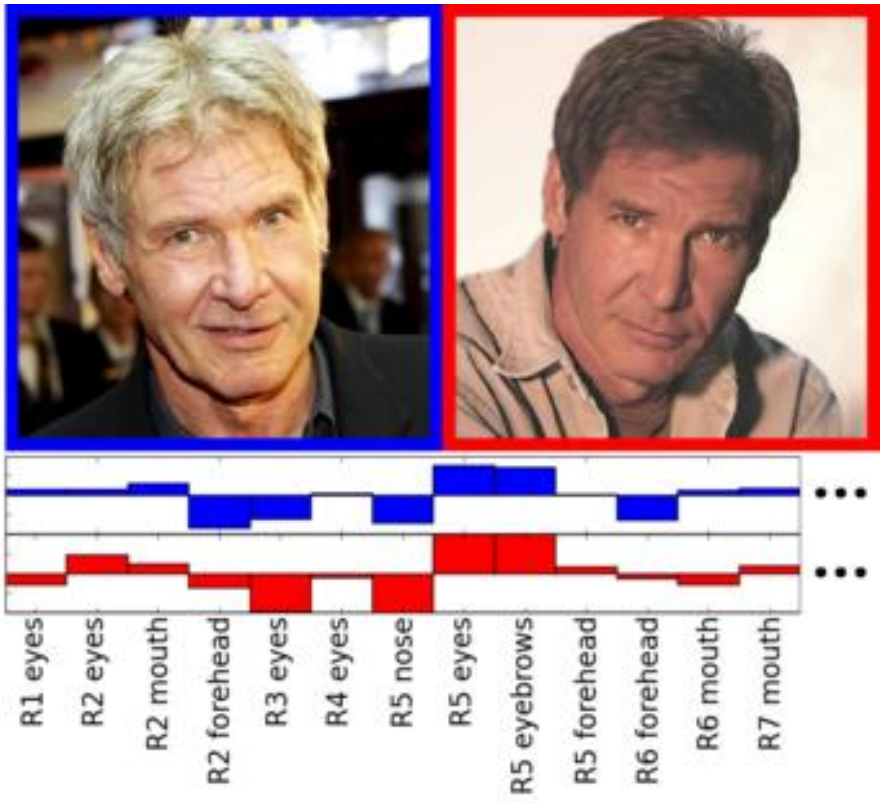
Limitations

- The direction of maximum variance is not always good for classification



Face Recognition





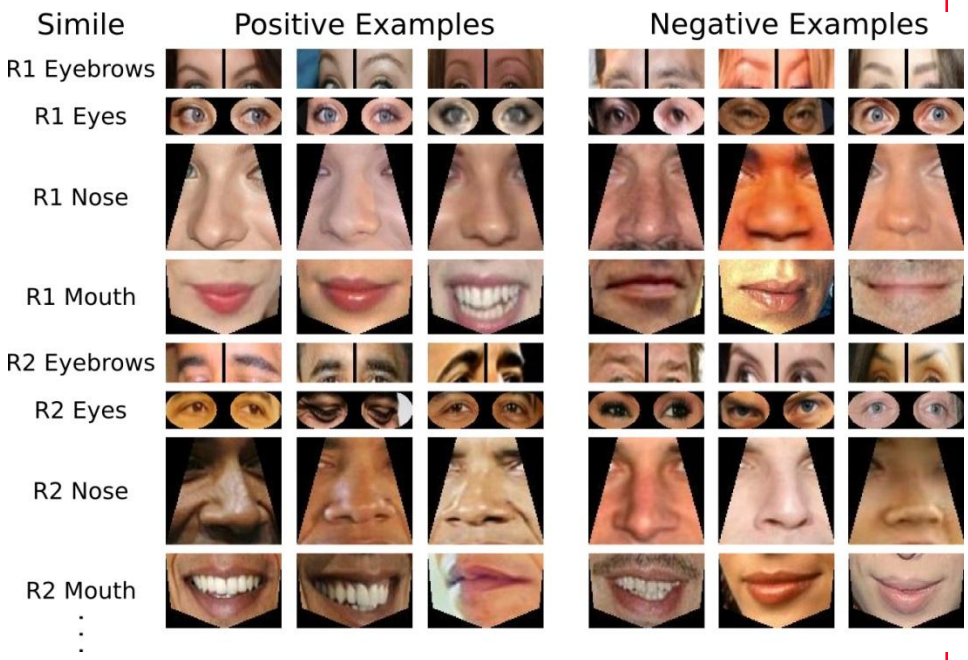
- N. Kumar, A. C. Berg, P. N. Belhumeur, and S. K. Nayar, **"Attribute and Simile Classifiers for Face Verification,"** ICCV 2009.

Face Recognition

Attributes for training



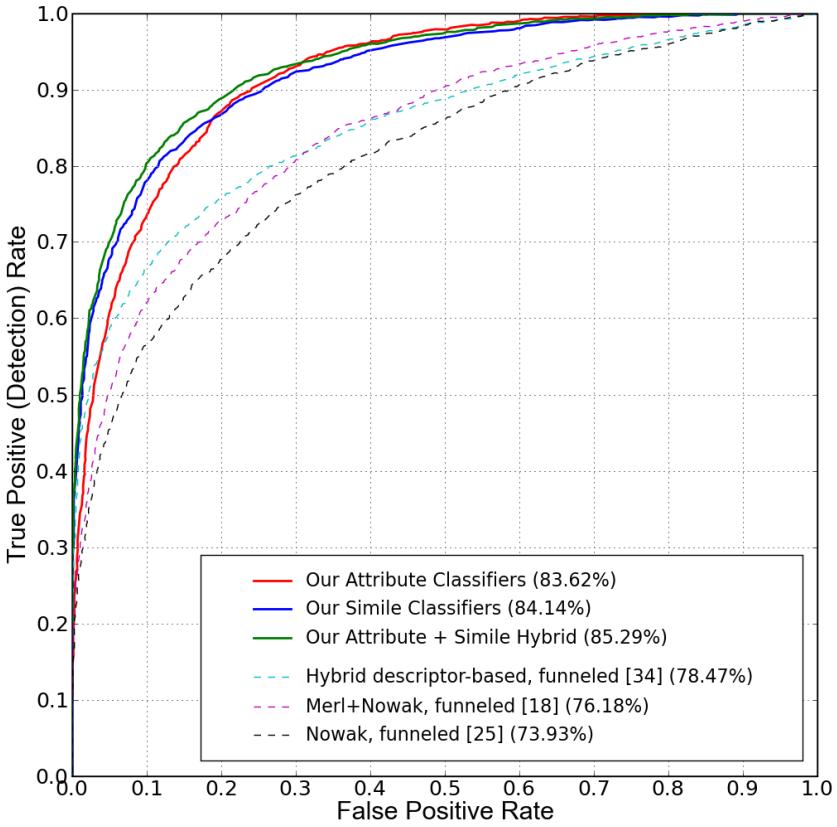
Similes for training



- N. Kumar, A. C. Berg, P. N. Belhumeur, and S. K. Nayar, **"Attribute and Simile Classifiers for Face Verification,"** ICCV 2009.

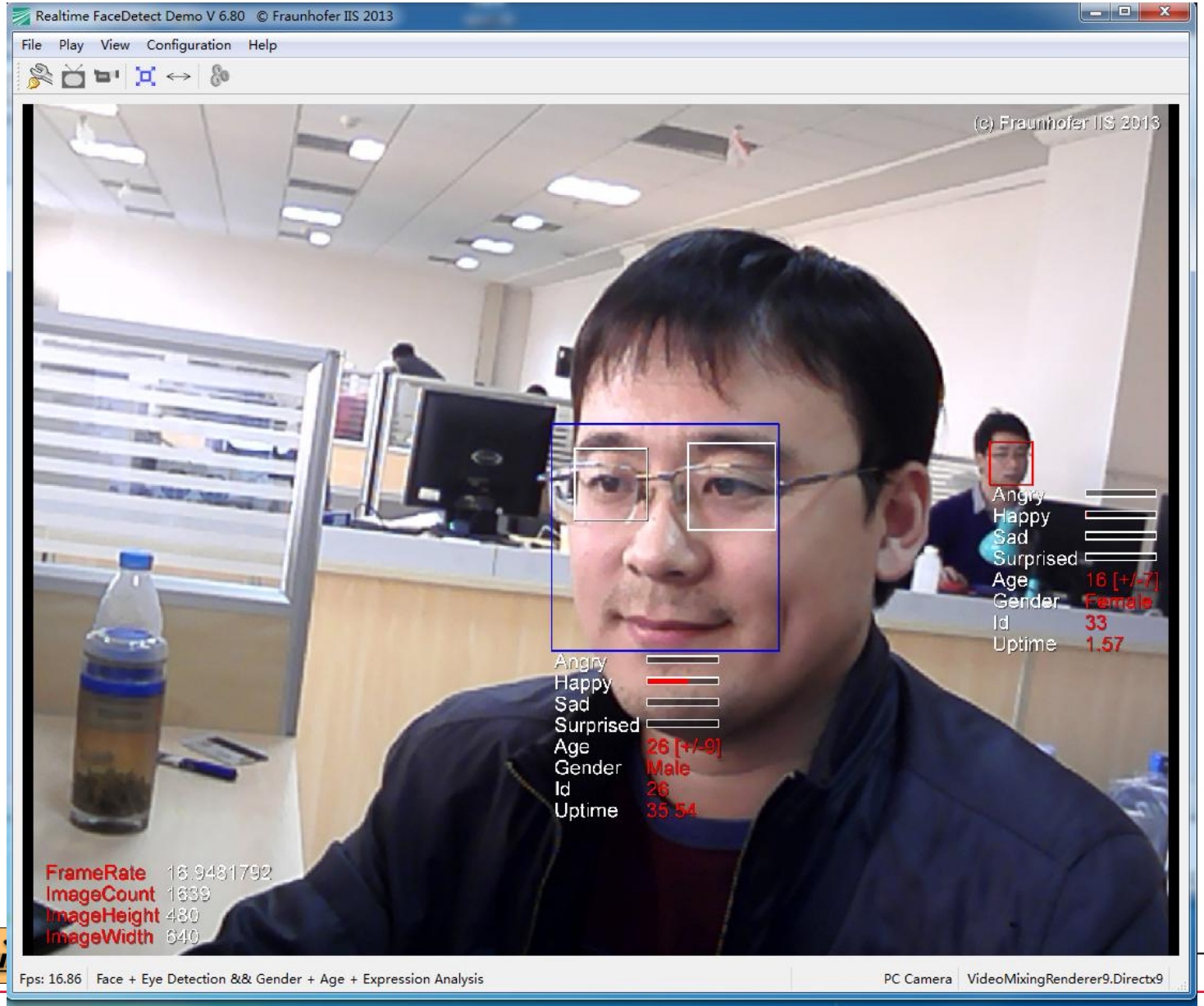
Face Recognition

Results on Labeled Faces in the Wild Dataset



- N. Kumar, A. C. Berg, P. N. Belhumeur, and S. K. Nayar, **"Attribute and Simile Classifiers for Face Verification,"** ICCV 2009.

Other tasks


Realtime FaceDetect Demo V 6.80 © Fraunhofer IIS 2013
File Play View Configuration Help
(c) Fraunhofer IIS 2013
Angry
Happy
Sad
Surprised
Age 16 [+/-7]
Gender Female
Id 33
Uptime 1.57
Angry
Happy
Sad
Surprised
Age 26 [+/-9]
Gender Male
Id 26
Uptime 35.54
FrameRate 16.9481792
ImageCount 1639
ImageHeight 480
ImageWidth 640
Fps: 16.86 Face + Eye Detection && Gender + Age + Expression Analysis
PC Camera VideoMixingRenderer9.Directx9

The end !